

000_INDICE_MAESTRO_DE_LA_BIBLIOT ECA

Sistema KAIROS

Catálogo general de la biblioteca interpretativa y operativa

1. Propósito del documento

Este documento funciona como el **índice maestro de toda la biblioteca KAIROS**. Su finalidad es ordenar, identificar y clasificar todos los documentos que forman parte del sistema, indicando con claridad:

- qué documento existe
- para qué sirve
- a qué serie pertenece
- de qué depende
- qué documentos alimenta
- en qué parte del motor interpretativo se utiliza
- cuál es su nivel de prioridad dentro del sistema

Este índice no contiene interpretaciones astrológicas en sí mismas. Su función es **estructural, organizativa y operativa**. Es el documento que permite comprender la arquitectura interna de la biblioteca como un sistema vivo, coherente y escalable.

2. Función dentro de KAIROS

El índice maestro cumple seis funciones centrales:

1. Ordenar la biblioteca

Permite ver de forma unificada todos los documentos existentes y su lugar dentro del sistema.

2. Evitar duplicidades

Ayuda a detectar contenidos repetidos, solapados o mal distribuidos entre series.

3. Mostrar jerarquía

Permite distinguir qué documentos son fundacionales, cuáles son interpretativos, cuáles son narrativos y cuáles son operativos.

4. Facilitar integración técnica

Sirve como referencia para migrar la biblioteca documental a formatos estructurados utilizables por la app, como Firestore o JSON versionado.

5. Facilitar mantenimiento

Permite versionar, ampliar o corregir la biblioteca sin perder el mapa global.

6. Servir como puerta de entrada al sistema

Toda persona que trabaje en KAIROS, sea en redacción, arquitectura o implementación, debe poder entender la biblioteca empezando por este documento.

3. Alcance

Este documento incluye:

- los documentos ya existentes de las series 100, 200, 300, 400 y 500
- los documentos operativos nuevos de la serie 000
- la relación básica entre documentos
- su uso dentro del motor interpretativo

No sustituye al mapa de dependencias ni al motor de orquestación.
Su función es catalogar, no ejecutar.

4. Principios de organización del índice

La biblioteca KAIROS se organiza según los siguientes principios:

4.1. Organización por series

Cada documento pertenece a una serie documental definida según su función dentro del sistema.

4.2. Un documento, una función principal

Aunque un documento pueda alimentar varias partes del sistema, debe tener una función principal clara.

4.3. Diferencia entre conocimiento y operación

No se debe mezclar en una misma categoría:

- teoría base
- combinaciones interpretativas
- formatos de salida
- reglas de ensamblaje
- lógica operativa del motor

4.4. Trazabilidad

Cada documento debe poder relacionarse con:

- sus dependencias
- los documentos que alimenta
- los módulos de salida donde se utiliza

4.5. Escalabilidad

El índice debe permitir ampliar la biblioteca sin romper el orden general.

5. Estructura general de la biblioteca KAIROS

La biblioteca KAIROS se divide en seis bloques documentales:

SERIE 000 — OPERATIVA Y GOBERNANZA

Documentos que organizan, conectan y hacen ejecutable el sistema.

SERIE 100 — FUNDAMENTOS

Documentos que definen el marco conceptual, el lenguaje y los principios del sistema.

SERIE 200 — BIBLIOTECA BASE

Documentos que contienen significados astrológicos fundamentales.

SERIE 300 — BIBLIOTECA RELACIONAL

Documentos que desarrollan combinaciones entre factores astrológicos.

SERIE 400 — SALIDA HUMANA

Documentos que definen cómo se expresa KAIROS al usuario.

SERIE 500 — REGLAS DE ENSAMBLAJE

Documentos que regulan cómo se integran, priorizan y limitan los contenidos.

6. Catálogo maestro de documentos

SERIE 000 — OPERATIVA Y GOBERNANZA

000_indice_maestro_de_la_biblioteca

Propósito: catalogar la totalidad de la biblioteca KAIROS y establecer su organización general.

Tipo: operativo-estructural.

Nivel: nuclear.

Depende de: conjunto completo de la biblioteca.

Alimenta a: todo el sistema.

Uso principal: orientación, mantenimiento, integración técnica y control documental.

001_mapa_de_dependencias

Propósito: mostrar qué documentos alimentan a cuáles y cómo fluye el conocimiento dentro del sistema.

Tipo: operativo-estructural.

Nivel: nuclear.

Depende de: 000 y catálogo real de documentos.

Alimenta a: 003, 004, 005.

Uso principal: arquitectura lógica y prevención de solapamientos.

002_esquema_de_variables_de_kairos

Propósito: definir todas las variables internas del sistema, su origen, su uso y su prioridad.

Tipo: operativo-técnico.

Nivel: nuclear.

Depende de: series 200, 300 y 500.

Alimenta a: 003, 004, integración en app.

Uso principal: traducción de astrología a datos estructurados.

003_motor_de_orquestacion_interpretativa_de_kairos

Propósito: definir cómo piensa KAIROS cuando recibe datos reales y genera una interpretación.

Tipo: operativo-lógico.

Nivel: crítico.

Depende de: 001, 002, 500.

Alimenta a: 004, motores JS, lógica de ensamblaje real.

Uso principal: cerebro interpretativo del sistema.

004_pipeline_de_generacion_de_respuesta

Propósito: definir el flujo operativo completo desde entrada de datos hasta salida visible en la app.

Tipo: operativo-funcional.

Nivel: crítico.

Depende de: 002, 003, 400, 500.

Alimenta a: implementación técnica y QA.

Uso principal: integración entre cálculo, biblioteca y UI.

005_reglas_de_QA_de_salida

Propósito: establecer criterios de control de calidad para impedir respuestas contradictorias, técnicas, fatalistas o pobres.

Tipo: operativo-control.

Nivel: crítico.

Depende de: 003, 004, 500.

Alimenta a: validación final de salida.

Uso principal: seguridad narrativa e interpretativa.

SERIE 100 — FUNDAMENTOS

100_criterios_generales_kairos

Propósito: definir el marco general del sistema KAIROS, su naturaleza, propósito y alcance.

Tipo: fundacional.

Nivel: nuclear.

Depende de: ninguno.

Alimenta a: toda la biblioteca.

110_glosario_base

Propósito: unificar lenguaje, términos y definiciones básicas del sistema.

Tipo: fundacional-semantico.

Nivel: alto.

Depende de: 100.

Alimenta a: 200, 300, 400, 500.

120_normas_de_redaccion

Propósito: establecer cómo debe escribirse el contenido del sistema.

Tipo: fundacional-narrativo.

Nivel: alto.

Depende de: 100, 110.

Alimenta a: 400, 500, 005.

130_tono_terapeutico_kairos

Propósito: definir el estilo humano, terapéutico, claro y no fatalista de la app.

Tipo: fundacional-narrativo.

Nivel: crítico.

Depende de: 100, 120.

Alimenta a: 400, 500, 005.

140_principios_de_interpretacion

Propósito: definir cómo debe interpretarse la información astrológica dentro de KAIROS.

Tipo: fundacional-lógico.

Nivel: crítico.

Depende de: 100, 110, 120, 130.

Alimenta a: 300, 500, 003.

SERIE 200 — BIBLIOTECA BASE

200_significado_planetas

Propósito: recoger el significado estructural de cada planeta dentro del sistema.

Tipo: semántico-base.

Nivel: alto.

Depende de: 110, 140.

Alimenta a: 300, 400, 003.

210_significado_signos

Propósito: definir el significado funcional de cada signo.

Tipo: semántico-base.

Nivel: alto.

Depende de: 110, 140.

Alimenta a: 300, 400, 003.

220_significado_casas

Propósito: definir el ámbito vital que representa cada casa.

Tipo: semántico-base.

Nivel: alto.

Depende de: 110, 140.

Alimenta a: 300, 400, 003.

230_fases_lunares

Propósito: definir el significado de las fases lunares como clima temporal.

Tipo: semántico-base.

Nivel: medio-alto.

Depende de: 140.

Alimenta a: 300, 400, diaria, semanal, mensual.

240_planetas_retrogradados

Propósito: explicar la cualidad interpretativa de la retrogradación.

Tipo: semántico-base.

Nivel: medio.

Depende de: 140.

Alimenta a: 300, 003.

250_planeta_del_dia

Propósito: definir el sentido interpretativo del planeta regente del día.

Tipo: semántico-base.

Nivel: medio.

Depende de: 140.

Alimenta a: 300, 400, diaria.

260_profeccion_anual

Propósito: definir la lógica de la profección anual como estructura del ciclo.

Tipo: semántico-estructural.

Nivel: crítico.

Depende de: 140.

Alimenta a: 300, 500, 003.

270_senor_del_ano

Propósito: definir el papel del Señor del Año dentro del sistema.

Tipo: semántico-estructural.

Nivel: crítico.

Depende de: 140, 260.

Alimenta a: 300, 500, 003.

280_elementos_y_modalidades

Propósito: definir elementos y modalidades como cualificadores de tono y energía.

Tipo: semántico-base.

Nivel: medio.

Depende de: 140.

Alimenta a: 300, matrix, apoyo interpretativo.

SERIE 300 — BIBLIOTECA RELACIONAL

300_planeta_en_signo

Propósito: desarrollar cómo se expresa un planeta según el signo en el que se encuentra.

Tipo: relacional.

Nivel: alto.

Depende de: 200, 210.

Alimenta a: 400, 003.

310_planeta_en_casa

Propósito: desarrollar cómo se expresa un planeta según la casa en la que actúa.

Tipo: relacional.

Nivel: alto.

Depende de: 200, 220.

Alimenta a: 400, 003.

320_senor_del_ano_por_planeta

Propósito: definir cómo cambia el tono del año según qué planeta rige el ciclo.

Tipo: relacional-estructural.

Nivel: crítico.

Depende de: 270, 200.

Alimenta a: anual, mensual, 003.

330_casa_activada_por_profeccion

Propósito: definir el significado de la casa anual activada.

Tipo: relacional-estructural.

Nivel: crítico.

Depende de: 260, 220.

Alimenta a: anual, mensual, 003.

340_luna_en_transito_por_signo

Propósito: definir el clima emocional corto según el signo actual de la Luna.

Tipo: relacional-dinámico.

Nivel: medio.

Depende de: 230, 210.

Alimenta a: diaria, semanal.

350_sol_actual_por_signo

Propósito: cualificar el tono estacional o de enfoque general según el signo solar actual.

Tipo: relacional-dinámico.

Nivel: medio.

Depende de: 210.

Alimenta a: mensual, semanal.

360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano

Propósito: integrar microclima diario con estructura anual cuando haya resonancia.

Tipo: relacional-combinado.

Nivel: alto.

Depende de: 250, 270, 340.

Alimenta a: diaria, semanal, 003.

SERIE 400 — SALIDA HUMANA

400_sintesis_breves

Propósito: definir la forma de construir síntesis interpretativas breves y claras.

Tipo: narrativo-salida.

Nivel: alto.

Depende de: 120, 130, 140.

Alimenta a: diaria, semanal, notificaciones.

410_advertencias

Propósito: establecer cómo formular advertencias sin fatalismo ni dramatización.

Tipo: narrativo-salida.

Nivel: alto.

Depende de: 130, 500.

Alimenta a: diaria, semanal, mensual.

420_acciones_recomendadas

Propósito: definir cómo traducir una lectura en orientación concreta y accionable.

Tipo: narrativo-salida.

Nivel: alto.

Depende de: 130, 140.

Alimenta a: todas las vistas.

430_preguntas_de_reflexion

Propósito: formular preguntas que ayuden al usuario a pensar su proceso.

Tipo: narrativo-salida.

Nivel: medio-alto.

Depende de: 130.

Alimenta a: diaria, semanal, diario.

440_lecturas_diarias

Propósito: definir la estructura y lógica de la lectura diaria.

Tipo: narrativo-funcional.

Nivel: crítico.

Depende de: 400, 410, 420, 430, 500.

Alimenta a: daily engine.

450_lecturas_semanales

Propósito: definir la estructura y lógica de la lectura semanal.

Tipo: narrativo-funcional.

Nivel: crítico.

Depende de: 400, 410, 420, 430, 500.

Alimenta a: weekly engine.

460_lecturas_mensuales

Propósito: definir la estructura y lógica de la lectura mensual.

Tipo: narrativo-funcional.

Nivel: crítico.

Depende de: 400, 410, 420, 430, 500.

Alimenta a: monthly engine.

470_matriz_cuerpo_mente_espiritu

Propósito: estructurar la lectura del estado interno en tres ejes comprensibles.

Tipo: narrativo-estructural.

Nivel: alto.

Depende de: 130, 140, 280, 500.

Alimenta a: matrix engine.

480_nivel_de_energia

Propósito: traducir la intensidad del momento a una escala comprensible y útil.

Tipo: narrativo-funcional.

Nivel: medio-alto.

Depende de: 500.

Alimenta a: radar, diaria, semanal.

490_textos_para_diario_personal

Propósito: generar prompts breves y profundos para escritura personal.

Tipo: narrativo-reflexivo.

Nivel: medio.

Depende de: 130, 430.

Alimenta a: journal mode.

SERIE 500 — REGLAS DE ENSAMBLAJE

500_reglas_de_prioridad_interpretativa

Propósito: definir qué factores pesan más dentro de KAIROS.

Tipo: ensamblaje-lógico.

Nivel: crítico.

Depende de: 140, 260, 270.

Alimenta a: 003, 004, QA.

501_reglas_de_conflicto

Propósito: definir cómo resolver tensiones o mensajes aparentemente opuestos.

Tipo: ensamblaje-lógico.

Nivel: crítico.

Depende de: 140, 500.

Alimenta a: 003, 005.

502_reglas_de_fallback

Propósito: definir qué hacer cuando no hay suficiente señal o faltan factores relevantes.

Tipo: ensamblaje-operativo.

Nivel: alto.

Depende de: 500.

Alimenta a: 003, 004, 005.

503_reglas_de_longitud

Propósito: regular la extensión de la respuesta según la vista.

Tipo: ensamblaje-formal.

Nivel: alto.

Depende de: 400.

Alimenta a: 004, 005.

504_reglas_de_personalizacion

Propósito: definir qué variables modulan la salida según el usuario o el contexto.

Tipo: ensamblaje-operativo.

Nivel: alto.

Depende de: 002, 400.

Alimenta a: 003, 004.

505_reglas_de_coherencia_entre_vistas

Propósito: asegurar que diaria, semanal, mensual y anual no se contradigan entre sí.

Tipo: ensamblaje-control.

Nivel: crítico.

Depende de: 500, 501, 503.

Alimenta a: 004, 005.

7. Clasificación funcional de documentos

Para facilitar la migración técnica, cada documento debe quedar etiquetado según su función principal.

7.1. Documentos fundacionales

Definen el marco conceptual del sistema.

Series: 100.

7.2. Documentos semánticos

Definen significados astrológicos base.

Series: 200.

7.3. Documentos relacionales

Definen combinaciones y cruces entre factores.

Series: 300.

7.4. Documentos de salida

Definen cómo se expresa la interpretación al usuario.

Series: 400.

7.5. Documentos de ensamblaje

Definen cómo se combinan y jerarquizan los factores.

Series: 500.

7.6. Documentos operativos

Definen cómo se vuelve ejecutable toda la biblioteca dentro de la app.

Series: 000.

8. Estados posibles de cada documento

Cada documento del sistema debe tener uno de estos estados:

- borrador
- en revisión
- validado
- activo
- obsoleto
- sustituido

Esto es importante para que KAIROS no acabe leyendo cosas antiguas como si fueran vigentes. Que eso pasa mucho en proyectos grandes y luego nadie sabe de dónde sale el monstruo.

9. Versionado recomendado

Cada documento debe registrar:

- código de documento
- versión
- fecha de actualización
- autoría o revisión
- estado
- cambios principales

Ejemplo:

Documento: 003_motor_de_orquestacion_interpretativa_de_kairos

Versión: 1.0

Estado: activo

Última actualización: [fecha]

Cambios principales: primera versión operativa del cerebro lógico de KAIROS

10. Uso del índice maestro dentro de la app

Este documento no debe ser consultado por el usuario final ni por la interfaz como contenido interpretativo. Su uso es interno.

Puede utilizarse para:

- organizar la migración documental a Firestore
- definir nodos por documento
- controlar versiones activas

- validar dependencias antes de activar nuevos módulos
 - auditar qué parte del sistema alimenta cada lectura
-

11. Criterio de mantenimiento

El índice maestro debe actualizarse siempre que ocurra cualquiera de estas situaciones:

- se cree un documento nuevo
 - se sustituya un documento
 - cambie el propósito de un documento
 - cambie una dependencia importante
 - se active una nueva vista en la app
 - se migre una parte de la biblioteca a una estructura técnica nueva
-

12. Cierre del documento

El índice maestro de la biblioteca KAIROS no interpreta el cielo ni redacta salidas. Su tarea es más humilde y más importante: **dar orden al sistema**.

Sin este documento, la biblioteca puede crecer.

Con este documento, la biblioteca puede crecer **sin desorganizarse**.

Y en un proyecto como KAIROS, eso no es un detalle administrativo: es una condición de supervivencia.

001_MAPA_DE_DEPENDENCIAS

Sistema KAIROS

Mapa lógico de relaciones entre documentos, funciones y niveles del sistema

1. Propósito del documento

Este documento define el **mapa de dependencias de la biblioteca KAIROS**. Su función es mostrar con claridad cómo se relacionan entre sí los distintos documentos del sistema, qué documentos alimentan a cuáles, qué tipo de dependencia existe entre ellos y cómo fluye el conocimiento desde los fundamentos conceptuales hasta la salida visible en la app.

El objetivo no es describir significados astrológicos ni redactar interpretaciones, sino **hacer visible la arquitectura lógica interna de la biblioteca** para que pueda ser:

- comprendida
- mantenida
- ampliada
- auditada
- integrada técnicamente en Antigravity, Firebase y Firestore

Este documento actúa como puente entre el índice maestro de la biblioteca y el motor de orquestación interpretativa. Si el índice maestro responde a la pregunta “**qué existe**”, el mapa de dependencias responde a la pregunta “**cómo se conecta**”.

2. Función dentro de KAIROS

El mapa de dependencias cumple las siguientes funciones:

1. Mostrar el flujo del conocimiento

Permite entender cómo un principio fundacional termina influyendo en una lectura diaria, semanal, mensual o anual.

2. Evitar ensamblajes arbitrarios

Impide que el sistema consulte documentos sin criterio o mezcle factores sin respetar jerarquía.

3. Delimitar responsabilidades documentales

Aclara qué documento aporta significado, cuál aporta tono, cuál aporta reglas y cuál aporta estructura operativa.

4. Facilitar la implementación técnica

Ayuda a traducir la biblioteca documental a nodos, colecciones, reglas y consultas dentro de Firestore o JSON estructurado.

5. Detectar huecos y solapamientos

Permite ver qué dependencias están cubiertas, cuáles están duplicadas y qué zonas del sistema todavía carecen de cierre operativo.

6. Preparar la orquestación real

Sirve de base para el documento 003, donde estas dependencias ya no se describen solo como relaciones, sino como lógica activa de decisión.

3. Alcance

Este documento incluye:

- dependencias entre series documentales
- dependencias entre documentos concretos
- distinción entre tipos de dependencia
- flujo de conocimiento hacia las salidas del sistema
- relación entre biblioteca documental y capa operativa

Este documento no sustituye:

- al índice maestro
- al esquema de variables
- al motor de orquestación
- al pipeline de respuesta
- a las reglas de QA

Su función es **cartografiar relaciones**, no ejecutar decisiones.

4. Principios generales del mapa de dependencias

El sistema KAIROS se rige por los siguientes principios de dependencia:

4.1. Toda salida depende de fundamentos

Ningún módulo de salida debe construirse directamente desde un significado aislado sin pasar por los principios generales de interpretación, tono y ensamblaje.

4.2. La biblioteca base no basta por sí sola

Los significados de planetas, signos, casas o fases no deben usarse de forma aislada como salida final. Necesitan ser combinados, jerarquizados y traducidos.

4.3. Las relaciones no sustituyen a la estructura

Las combinaciones relacionales enriquecen el sistema, pero no deben desplazar los factores estructurales del ciclo, como la profección anual o el Señor del Año.

4.4. La salida humana no inventa contenido

Los documentos de la serie 400 traducen y formatean. No redefinen el significado ni alteran la jerarquía del sistema.

4.5. Las reglas de ensamblaje gobiernan la integración

La serie 500 no añade contenido astrológico nuevo. Su papel es ordenar, priorizar, limitar y hacer coherente el ensamblaje.

4.6. La serie operativa convierte la biblioteca en sistema ejecutable

La serie 000 no interpreta el cielo. Organiza, modela y hace implementable toda la biblioteca.

5. Tipos de dependencia

Para evitar ambigüedad, KAIROS debe distinguir varios tipos de dependencia. No todas las relaciones entre documentos son del mismo tipo.

5.1. Dependencia fundacional

Se produce cuando un documento necesita otro para sostener su marco conceptual, su lenguaje o su enfoque general.

Ejemplo:

140_principios_de_interpretacion depende fundacionalmente de 100_criterios_generales_kairos, 110_glosario_base, 120_normas_de_redaccion y 130_tono_terapeutico_kairos.

5.2. Dependencia semántica

Se produce cuando un documento necesita significados base para construir una interpretación o combinación.

Ejemplo:

300_planeta_en_signo depende semánticamente de 200_significado_planetas y 210_significado_signos.

5.3. Dependencia relacional

Se produce cuando un documento necesita otro para construir cruces o combinaciones entre factores.

Ejemplo:

360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano depende relacionalmente de 250_planeta_del_dia, 270_senor_del_ano y 340_luna_en_transito_por_signo.

5.4. Dependencia narrativa

Se produce cuando un documento necesita otro para poder expresarse correctamente al usuario en tono, estilo o formato.

Ejemplo:

440_lecturas_diarias depende narrativamente de 120_normas_de_redaccion, 130_tono_terapeutico_kairos, 400_sintesis_breves, 410_advertencias, 420_acciones_recomendadas y 430_preguntas_de_reflexion.

5.5. Dependencia de ensamblaje

Se produce cuando un documento necesita reglas de prioridad, conflicto, fallback, personalización o longitud para poder integrarse sin contradicción.

Ejemplo:

460_lecturas_mensuales depende de ensamblaje de
500_reglas_de_prioridad_interpretativa, 501_reglas_de_conflicto,
502_reglas_de_fallback, 503_reglas_de_longitud,
504_reglas_de_personalizacion y 505_reglas_de_coherencia_entre_vistas.

5.6. Dependencia operativa

Se produce cuando un documento necesita otro para ser implementado en lógica real, variables, pipeline o arquitectura de sistema.

Ejemplo:

003_motor_de_orquestacion_interpretativa_de_kairos depende
operativamente de 001_mapa_de_dependencias,
002_esquema_de_variables_de_kairos y la serie 500.

6. Arquitectura global de dependencias

La biblioteca KAIROS se organiza en un flujo descendente de seis capas:

CAPA 1 — Fundamentos

Serie 100

Define qué es KAIROS, cómo nombra, cómo interpreta y cómo habla.

CAPA 2 — Significados base

Serie 200

Define el significado funcional de los factores astrológicos principales.

CAPA 3 — Relaciones interpretativas

Serie 300

Define cómo se combinan los factores entre sí.

CAPA 4 — Reglas de ensamblaje

Serie 500

Define cómo se priorizan, integran y limitan esos factores.

CAPA 5 — Operativa del sistema

Serie 000

Define cómo se convierten esas relaciones en motor, variables, pipeline y control de calidad.

CAPA 6 — Salida humana

Serie 400

Define cómo se traduce el resultado a bloques legibles, terapéuticos, claros y modulares.

7. Flujo general del conocimiento

El flujo lógico general de KAIROS debe entenderse así:

100 Fundamentos

↓

200 Biblioteca base

↓

300 Biblioteca relacional

↓

500 Reglas de ensamblaje

↓

000 Serie operativa

↓

400 Salida humana

↓

Interfaz de la app

Este flujo no significa que la salida humana “dependa menos” que la operativa. Significa que, dentro del sistema, la salida solo puede ser sólida si está sostenida por una cadena previa de significado, estructura, jerarquía y control.

8. Dependencias entre series

8.1. Serie 100 — FUNDAMENTOS

La serie 100 es la raíz conceptual del sistema.
Todos los bloques posteriores dependen de ella de forma directa o indirecta.

Alimenta directamente a:

- serie 200
- serie 300
- serie 400
- serie 500
- serie 000

Recibe dependencia de:

- ninguna otra serie

Tipo de función:

- marco conceptual
- lenguaje común
- criterios interpretativos
- tono y límites

Riesgo si falta o está mal resuelta:

- incoherencia global
 - contradicción de estilo
 - pérdida de criterio interpretativo
 - desorden entre teoría, técnica y salida
-

8.2. Serie 200 — BIBLIOTECA BASE

La serie 200 depende de la serie 100 y funciona como base semántica del sistema.

Depende de:

- serie 100

Alimenta directamente a:

- serie 300
- serie 400
- serie 500
- serie 000

Tipo de función:

- significado de factores fundamentales
- definición de lenguaje astrológico funcional
- sostén básico de interpretación

Riesgo si falta o está mal resuelta:

- combinaciones pobres o inconsistentes
 - exceso de intuición narrativa
 - falta de trazabilidad en el significado
-

8.3. Serie 300 — BIBLIOTECA RELACIONAL

La serie 300 depende de la serie 200 y de la serie 100. Es la capa donde los significados dejan de estar aislados y empiezan a funcionar como sistema.

Depende de:

- serie 100
- serie 200

Alimenta directamente a:

- serie 400
- serie 500
- serie 000

Tipo de función:

- combinaciones entre factores
- integración contextual
- traducción de datos a núcleos de sentido

Riesgo si falta o está mal resuelta:

- lecturas demasiado genéricas
 - ensamblajes mecánicos
 - pérdida de matiz y precisión interpretativa
-

8.4. Serie 400 — SALIDA HUMANA

La serie 400 no debe entenderse como una capa “decorativa”. Es la capa que traduce el sistema a experiencia humana, pero depende de casi todo lo anterior.

Depende de:

- serie 100
- serie 200
- serie 300
- serie 500
- serie 000

Alimenta directamente a:

- daily engine
- weekly engine
- monthly engine
- matrix engine
- journal mode
- notification mode

Tipo de función:

- traducción narrativa
- estructura modular de la lectura
- interfaz verbal del sistema

Riesgo si falta o está mal resuelta:

- salida fría, confusa o técnica
 - contradicción entre bloques
 - deriva hacia horóscopo o espiritualidad vacía
-

8.5. Serie 500 — REGLAS DE ENSAMBLAJE

La serie 500 regula cómo se integra todo lo anterior. No añade significados nuevos; da orden, prioridad y límites.

Depende de:

- serie 100
- serie 200
- serie 300

Alimenta directamente a:

- serie 400
- serie 000
- motor interpretativo

- QA de salida

Tipo de función:

- jerarquía
- conflicto
- fallback
- longitud
- personalización
- coherencia entre vistas

Riesgo si falta o está mal resuelta:

- contradicciones internas
 - sobrereacción ante factores menores
 - salidas largas, caóticas o infladas
 - inconsistencia entre diaria, semanal y mensual
-

8.6. Serie 000 — OPERATIVA Y GOBERNANZA

La serie 000 no reemplaza a la biblioteca interpretativa; la vuelve utilizable por un sistema real.

Depende de:

- serie 100
- serie 200
- serie 300
- serie 400
- serie 500

Alimenta directamente a:

- arquitectura técnica
- Firestore
- motores JS
- orquestación real
- QA
- integración UI

Tipo de función:

- modelado del sistema
- definición de variables
- flujo operativo

- control de calidad
- documentación de integración

Riesgo si falta o está mal resuelta:

- la biblioteca existe pero no piensa
- el sistema calcula, pero no orquesta
- la app genera bloques, pero no inteligencia coherente

9. Dependencias críticas entre documentos

A continuación se detallan las dependencias principales entre los documentos nucleares del sistema.

9.1. Dependencias dentro de la serie 100

100_criterios_generales_kairos

No depende de ningún documento previo.
Es la raíz del sistema.

110_glosario_base

Depende de:

- 100_criterios_generales_kairos

120_normas_de_redaccion

Depende de:

- 100_criterios_generales_kairos
- 110_glosario_base

130_tono_terapeutico_kairos

Depende de:

- 100_criterios_generales_kairos
- 120_normas_de_redaccion

140_principios_de_interpretacion

Depende de:

- 100_criterios_generales_kairos
- 110_glosario_base
- 120_normas_de_redaccion
- 130_tono_terapeutico_kairos

El documento 140 es el principal puente entre fundamentos y lógica interpretativa.

9.2. Dependencias dentro de la serie 200

200_significado_planetas

Depende de:

- 110_glosario_base
- 140_principios_de_interpretacion

210_significado_signos

Depende de:

- 110_glosario_base
- 140_principios_de_interpretacion

220_significado_casas

Depende de:

- 110_glosario_base
- 140_principios_de_interpretacion

230_fases_lunares

Depende de:

- 140_principios_de_interpretacion

240_planetas_retrogradados

Depende de:

- 140_principios_de_interpretacion

250_planeta_del_dia

Depende de:

- 140_principios_de_interpretacion

260_profeccion_anual

Depende de:

- 140_principios_de_interpretacion

270_senor_del_ano

Depende de:

- 140_principios_de_interpretacion
- 260_profeccion_anual

280_elementos_y_modalidades

Depende de:

- 140_principios_de_interpretacion
-

9.3. Dependencias dentro de la serie 300

300_planeta_en_signo

Depende de:

- 200_significado_planetas
- 210_significado_signos

310_planeta_en_casa

Depende de:

- 200_significado_planetas
- 220_significado_casas

320_senor_del_ano_por_planeta

Depende de:

- 270_senor_del_ano
- 200_significado_planetas

330_casa_activada_por_profeccion

Depende de:

- 260_profeccion_anual
- 220_significado_casas

340_luna_en_transito_por_signo

Depende de:

- 230_fases_lunares
- 210_significado_signos

350_sol_actual_por_signo

Depende de:

- 210_significado_signos

360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano

Depende de:

- 250_planeta_del_dia
 - 270_senor_del_ano
 - 340_luna_en_transito_por_signo
-

9.4. Dependencias dentro de la serie 400

400_sintesis_breves

Depende de:

- 120_normas_de_redaccion
- 130_tono_terapeutico_kairos
- 140_principios_de_interpretacion

410_advertencias

Depende de:

- 130_tono_terapeutico_kairos
- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto

420_acciones_recomendadas

Depende de:

- 130_tono_terapeutico_kairos
- 140_principios_de_interpretacion

430_preguntas_de_reflexion

Depende de:

- 130_tono_terapeutico_kairos

440_lecturas_diarias

Depende de:

- 400_sintesis_breves
- 410_advertencias
- 420_acciones_recomendadas
- 430_preguntas_de_reflexion
- 250_planeta_del_dia
- 230_fases_lunares
- 340_luna_en_transito_por_signo
- 360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano
- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto
- 502_reglas_de_fallback
- 503_reglas_de_longitud
- 505_reglas_de_coherencia_entre_vistas

450_lecturas_semanales

Depende de:

- 400_sintesis_breves
- 410_advertencias
- 420_acciones_recomendadas
- 430_preguntas_de_reflexion
- 230_fases_lunares
- 340_luna_en_transito_por_signo
- 350_sol_actual_por_signo
- 360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano
- tránsitos priorizados del sistema
- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto
- 502_reglas_de_fallback
- 503_reglas_de_longitud
- 505_reglas_de_coherencia_entre_vistas

460_lecturas_mensuales

Depende de:

- 400_sintesis_breves
- 410_advertencias
- 420_acciones_recomendadas
- 320_senor_del_ano_por_planeta
- 330_casa_activada_por_profesion
- 350_sol_actual_por_signo
- tránsitos mensuales priorizados
- lunaciones relevantes
- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto
- 502_reglas_de_fallback
- 503_reglas_de_longitud
- 504_reglas_de_personalizacion
- 505_reglas_de_coherencia_entre_vistas

470_matriz_cuerpo_mente_espiritu

Depende de:

- 130_tono_terapeutico_kairos
- 140_principios_de_interpretacion
- 280_elementos_y_modalidades
- 500_reglas_de_prioridad_interpretativa
- 504_reglas_de_personalizacion

480_nivel_de_energia

Depende de:

- tránsitos priorizados
- fase lunar
- clima lunar
- 500_reglas_de_prioridad_interpretativa
- 503_reglas_de_longitud

490_textos_para_diario_personal

Depende de:

- 130_tono_terapeutico_kairos
 - 430_preguntas_de_reflexion
 - 505_reglas_de_coherencia_entre_vistas
-

9.5. Dependencias dentro de la serie 500

500_reglas_de_prioridad_interpretativa

Depende de:

- 140_principios_de_interpretacion
- 260_profeccion_anual
- 270_senor_del_ano
- lógica estructural del sistema

501_reglas_de_conflicto

Depende de:

- 140_principios_de_interpretacion
- 500_reglas_de_prioridad_interpretativa

502_reglas_de_fallback

Depende de:

- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto

503_reglas_de_longitud

Depende de:

- 400_sintesis_breves
- vistas del sistema

504_reglas_de_personalizacion

Depende de:

- 002_esquema_de_variables_de_kairos
- 400_sintesis_breves
- estructura de salidas

505_reglas_de_coherencia_entre_vistas

Depende de:

- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto
- 503_reglas_de_longitud

9.6. Dependencias dentro de la serie 000

000_indice_maestro_de_la_biblioteca

Depende de:

- catálogo completo de la biblioteca

001_mapa_de_dependencias

Depende de:

- 000_indice_maestro_de_la_biblioteca
- estructura real de la biblioteca

002_esquema_de_variables_de_kairos

Depende de:

- serie 200
- serie 300
- serie 500
- arquitectura real de la app

003_motor_de_orquestacion_interpretativa_de_kairos

Depende de:

- 001_mapa_de_dependencias
- 002_esquema_de_variables_de_kairos
- serie 500
- motores astronómicos y de cálculo

004_pipeline_de_generacion_de_respuesta

Depende de:

- 002_esquema_de_variables_de_kairos
- 003_motor_de_orquestacion_interpretativa_de_kairos
- serie 400
- serie 500
- arquitectura de app y backend

005_reglas_de_QA_de_salida

Depende de:

- 003_motor_de_orquestacion_interpretativa_de_kairos
- 004_pipeline_de_generacion_de_respuesta
- 130_tono_terapeutico_kairos
- serie 500

10. Dependencias por tipo de salida

Además de las dependencias documentales, KAIROS debe registrar dependencias por modo de lectura.

10.1. Lectura anual

Factores dominantes:

- profección anual
- casa activada
- Señor del Año
- condición natal del Señor del Año
- tránsitos mayores de apoyo

Dependencias principales:

- 260_profeccion_anual
- 270_senor_del_ano
- 320_senor_del_ano_por_planeta
- 330_casa_activada_por_profeccion
- 500_reglas_de_prioridad_interpretativa
- 501_reglas_de_conflicto
- 504_reglas_de_personalizacion
- 505_reglas_de_coherencia_entre_vistas

Observación:

La lectura anual es la estructura superior del sistema.

Las demás vistas no deben contradecirla, solo desarrollarla o modularla.

10.2. Lectura mensual

Factores dominantes:

- estructura anual vigente
- tránsitos mensuales relevantes

- sol actual
- lunaciones
- activaciones de la casa profectada o del Señor del Año

Dependencias principales:

- 320_senor_del_ano_por_planeta
- 330_casa_activada_por_profeccion
- 350_sol_actual_por_signo
- documentos de lunaciones relevantes
- reglas de prioridad y conflicto
- módulos narrativos mensuales

Observación:

La lectura mensual debe conectar el clima dinámico del mes con la estructura anual sin diluirla.

10.3. Lectura semanal

Factores dominantes:

- continuidad del mes
- tránsitos activos de la semana
- clima lunar
- ritmo de foco o contraste

Dependencias principales:

- 340_luna_en_transito_por_signo
- 350_sol_actual_por_signo
- 360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano
- reglas de conflicto y longitud
- módulos de síntesis, advertencia, acción y reflexión

Observación:

La semanal actúa como puente entre el proceso mensual y el movimiento diario.

10.4. Lectura diaria

Factores dominantes:

- Luna actual
- planeta del día
- fase lunar
- resonancia con la estructura anual
- tránsito activo si realmente tiene peso

Dependencias principales:

- 230_fases_lunares
- 250_planeta_del_dia
- 340_luna_en_transito_por_signo
- 360_cruces_entre_luna_planeta_del_dia_y_senor_del_ano
- reglas de fallback
- reglas de prioridad
- módulos breves de salida

Observación:

La lectura diaria no debe convertirse en una mini lectura anual.

Su función es modular el clima del día sin perder conexión con la estructura mayor.

10.5. Matrix cuerpo-mente-espíritu

Factores dominantes:

- balance energético general
- distribución de tensiones
- tono emocional y mental
- criterios de organización interna del sistema

Dependencias principales:

- 280_elementos_y_modalidades
- reglas de prioridad
- reglas de personalización
- definición interna de ejes cuerpo, mente y espíritu

Observación:

La matrix no es una lectura paralela separada del sistema, sino otra forma de presentar el estado interpretativo.

10.6. Diario personal

Factores dominantes:

- síntesis emocional del momento
- eje reflexivo principal
- coherencia con la vista del día o de la semana

Dependencias principales:

- 430_preguntas_de_reflexion
- 490_textos_para_diario_personal
- tono terapéutico
- coherencia entre vistas

Observación:

El diario personal no debe introducir contenido nuevo contradictorio. Debe profundizar el núcleo ya interpretado.

11. Dependencias entre biblioteca documental y arquitectura técnica

El mapa de dependencias también debe ayudar a traducir la biblioteca a una estructura útil para la app.

11.1. Dependencias con motores astronómicos y de cálculo

Los motores JS existentes aportan datos, no significado narrativo final. Su función es alimentar la biblioteca operativa.

Motores relevantes:

- `chart_engine.js`
- `ascendant_engine.js`
- `astronomy.js`
- `projection_engine.js`
- `transit_engine.js`
- `planetary_engine.js`
- `daily_engine.js`
- `weekly_engine.js`
- `monthly_engine.js`
- `radar_engine.js`
- `history_engine.js`

Relación de dependencia:

- los motores de cálculo generan variables
- las variables alimentan el esquema 002
- el 002 alimenta el 003
- el 003 consulta reglas y nodos documentales
- el 004 transforma eso en pipeline ejecutable
- el 005 valida el resultado

11.2. Dependencias con Firestore

La biblioteca maestra humana puede existir en PDF o Drive, pero la app necesita una versión estructurada y consultable.

Por tanto:

- los documentos fuente alimentan nodos estructurados
- los nodos estructurados alimentan el motor
- el motor no debe depender de PDFs en tiempo real
- Firestore debe almacenar versiones consultables, no solo archivos largos

11.3. Dependencias con la UI

La interfaz no debe decidir significado.

La UI solo debe mostrar módulos ya ensamblados.

Por tanto:

- la UI depende del pipeline de salida
- el pipeline depende del motor
- el motor depende del esquema de variables y de la biblioteca estructurada
- la biblioteca estructurada depende de la biblioteca documental maestra

12. Reglas de lectura del mapa

Para que este documento sea realmente útil, debe leerse con estas reglas:

12.1. Una dependencia no equivale a consulta permanente

Que un documento dependa de otro no significa que deba consultarse entero en cada respuesta. Significa que necesita su contenido como base o marco.

12.2. Las dependencias no siempre son simétricas

Si `440_lecturas_diarias` depende de `340_luna_en_transito_por_signo`, eso no significa que `340` dependa de `440`.

12.3. No todas las dependencias tienen el mismo peso

Hay dependencias nucleares y dependencias secundarias.
El motor interpretativo deberá ponderarlas.

12.4. El mapa no sustituye al scoring

Este documento muestra relaciones.

La jerarquía exacta de pesos se definirá en el 003 y en las reglas de prioridad de la serie 500.

13. Criterio de mantenimiento

Este documento debe actualizarse cuando ocurra cualquiera de estas situaciones:

- se añade un documento nuevo a cualquier serie
- cambia el propósito de un documento existente
- cambia una relación fuerte entre documentos
- se crea una nueva vista de salida
- se modifica la arquitectura real de la app
- se sustituye un motor de cálculo
- se migra la biblioteca a una nueva estructura técnica

14. Cierre del documento

El mapa de dependencias de KAIROS no es burocracia. Es una pieza central del sistema.

Una biblioteca puede ser rica y aun así fallar si no está claro qué depende de qué.

Un motor puede calcular bien y aun así entregar ruido si no sabe qué estructura respetar.

Una app puede verse bonita y aun así pensar mal si mezcla capas sin criterio.

Este documento existe para evitar eso.

Su función es dejar claro que en KAIROS:

- los fundamentos sostienen
- los significados alimentan
- las relaciones matizan
- las reglas ordenan
- la operativa ejecuta
- y la salida humana traduce

En ese orden. No al revés.

002_ESQUEMA_DE_VARIABLES_DE_KAIROS

Sistema KAIROS

Definición estructural de variables, categorías, origen, prioridad y uso operativo

1. Propósito del documento

Este documento define el **esquema general de variables de KAIROS**. Su objetivo es traducir la arquitectura astrológica e interpretativa del sistema a una estructura de datos clara, consistente y utilizable dentro de la app.

KAIROS no puede operar directamente sobre conceptos amplios o textos de biblioteca. Para generar respuestas coherentes, necesita trabajar con variables bien definidas que permitan:

- almacenar datos del usuario
- calcular contexto astral
- identificar factores activos
- asignar jerarquía interpretativa
- seleccionar nodos de biblioteca
- ensamblar salidas modulares
- verificar coherencia y calidad final

Este documento no interpreta ni redacta lecturas. Su función es **convertir astrología, contexto temporal y lógica operativa en un modelo de datos estable**.

2. Función dentro de KAIROS

El esquema de variables cumple las siguientes funciones:

1. Traducir el sistema astrológico a estructura computable

Permite que la app no dependa de intuiciones ni de textos sueltos, sino de datos con nombre, tipo y uso definido.

2. Separar niveles de información

Distingue entre datos natales, datos del ciclo, datos del momento, variables derivadas y variables de salida.

3. Facilitar el trabajo del motor interpretativo

El motor de orquestación no debe “adivinar” qué tiene entre manos. Debe recibir variables consistentes.

4. Facilitar integración técnica

Este documento sirve como base para modelado en Firestore, objetos JS, funciones de cálculo y queries de biblioteca.

5. Evitar caos semántico

Ayuda a impedir que una misma idea aparezca con nombres distintos en varios módulos o que una variable se use con sentidos cambiantes.

6. Preparar versionado y QA

Permite rastrear qué variable alimenta qué módulo y detectar errores cuando una salida falla.

3. Alcance

Este documento incluye:

- categorías de variables
- definición funcional de cada grupo
- nombre interno recomendado
- origen de la variable
- tipo de dato
- obligatoriedad
- frecuencia de actualización
- prioridad interpretativa
- uso dentro del motor y de la salida

Este documento no sustituye:

- el motor de orquestación
- el pipeline de respuesta
- la lógica de scoring
- las reglas de QA

Su función es definir **qué información existe en el sistema y cómo debe organizarse**.

4. Principios de diseño del esquema de variables

KAIROS debe regirse por estos principios al definir sus variables:

4.1. Una variable, una función principal

Cada variable debe tener un propósito claro. Si una misma variable intenta servir para demasiadas cosas, acaba volviéndose ambigua y peligrosa.

4.2. Separación de capas

No deben mezclarse en el mismo bloque:

- datos de identidad
- datos natales
- datos temporales
- datos derivados
- datos de salida
- metadatos de control

4.3. Trazabilidad

Toda variable importante debe poder responder a estas preguntas:

- de dónde viene
- quién la calcula
- para qué sirve
- en qué vistas se usa
- con qué prioridad trabaja

4.4. Compatibilidad con app real

El esquema debe ser legible para alguien no técnico, pero también utilizable dentro de:

- Firestore
- objetos JS
- motores de cálculo
- motores de salida
- QA automático

4.5. Compatibilidad con crecimiento

El sistema debe poder incorporar variables nuevas sin romper la estructura general.

4.6. Prioridad interpretativa explícita

No basta con definir variables. También hay que distinguir si son:

- estructurales
- contextuales
- modulares
- auxiliares
- de control

5. Estructura general del sistema de variables

El esquema de variables de KAIROS se organiza en ocho bloques:

1. Variables de identidad y configuración del usuario
2. Variables natales estructurales
3. Variables del ciclo anual

4. Variables temporales dinámicas
5. Variables de tránsitos y activaciones
6. Variables derivadas por el motor interpretativo
7. Variables de salida
8. Variables de control, trazabilidad y QA

Cada uno de estos bloques cumple una función específica dentro del sistema.

6. BLOQUE A — VARIABLES DE IDENTIDAD Y CONFIGURACIÓN DEL USUARIO

6.1. Función del bloque

Este bloque reúne los datos necesarios para identificar al usuario, personalizar la experiencia básica y situar correctamente el cálculo astral.

No contiene interpretación en sí misma, pero sin este bloque el resto del sistema no puede funcionar con precisión suficiente.

6.2. Variables recomendadas

user_id

Función: identificador único interno del usuario.

Tipo: string.

Origen: sistema / Firebase Auth / Firestore.

Obligatoria: sí.

Frecuencia de actualización: fija.

Uso principal: persistencia, trazabilidad, historial, personalización.

display_name

Función: nombre visible del usuario dentro de la app.

Tipo: string.

Origen: perfil de usuario.

Obligatoria: no.

Frecuencia de actualización: ocasional.

Uso principal: personalización ligera de la experiencia.

lang

Función: idioma activo de la app y de la salida interpretativa.

Tipo: string.

Valores esperados: `es`, `en`, u otros futuros.

Origen: configuración de usuario o sistema.

Obligatoria: sí.

Uso principal: selección de nodos de salida y renderizado narrativo.

timezone

Función: zona horaria del usuario.

Tipo: string.

Ejemplo: `Europe/Madrid`.

Origen: configuración del sistema o del usuario.

Obligatoria: sí.

Uso principal: cálculo correcto de fecha activa, Luna, fase lunar, planeta del día y vistas temporales.

birth_date

Función: fecha de nacimiento del usuario.

Tipo: date o string normalizada.

Obligatoria: sí.

Uso principal: carta natal, edad, profección anual.

birth_time

Función: hora de nacimiento del usuario.

Tipo: string u hora normalizada.

Obligatoria: sí, salvo modo incompleto.

Uso principal: ascendente, casas, ángulos, precisión natal.

birth_place_name

Función: nombre del lugar de nacimiento.

Tipo: string.

Obligatoria: sí.

Uso principal: referencia humana y trazabilidad.

birth_lat

Función: latitud del lugar de nacimiento.

Tipo: number.

Obligatoria: sí.

Uso principal: cálculo astronómico.

birth_lng

Función: longitud del lugar de nacimiento.

Tipo: number.

Obligatoria: sí.

Uso principal: cálculo astronómico.

birth_data_precision

Función: indicar nivel de fiabilidad de la hora de nacimiento.

Tipo: string.

Valores sugeridos: `exact`, `approximate`, `unknown`.

Obligatoria: sí.

Uso principal: control de confianza interpretativa.

user_plan

Función: nivel de acceso del usuario dentro de la app.

Tipo: string.

Valores sugeridos: `free`, `premium`, `advanced`.

Obligatoria: sí.

Uso principal: controlar vistas y profundidad de salida.

user_preferences

Función: preferencias de experiencia del usuario.

Tipo: object.

Posibles campos: longitud preferida, notificaciones activas, vista favorita.

Obligatoria: no.

Uso principal: personalización secundaria.

7. BLOQUE B — VARIABLES NATALES ESTRUCTURALES

7.1. Función del bloque

Este bloque contiene la estructura natal del usuario. Es la base estable sobre la que se interpretan el año, el mes, la semana y el día.

Estas variables no cambian con el tiempo. Son el contexto fijo del sistema.

7.2. Variables recomendadas

natal_chart_ready

Función: indicar si la carta natal ha sido calculada correctamente.

Tipo: boolean.

Obligatoria: sí.

Uso principal: validación previa del motor.

natal_sun_sign

natal_moon_sign

natal_asc_sign

Función: signos principales de identidad base.

Tipo: string.

Obligatoria: sí.

Uso principal: personalización, presentación general, capas de interpretación.

natal_mc_sign

Función: signo del Medio Cielo.

Tipo: string.

Obligatoria: recomendable.

Uso principal: capas vocacionales o de dirección.

natal_planet_positions

Función: posiciones natales completas de los planetas.

Tipo: object o array de objetos.

Uso principal: cálculos relacionales y tránsitos.

Estructura recomendada:

```
{  
  "Sun": {"sign": "Gemini", "degree": 12.4, "house": 11, "retrograde": false},  
  "Moon": {"sign": "Aries", "degree": 3.1, "house": 9, "retrograde": false}  
}
```

natal_houses

Función: signos de cúspides y distribución de casas.

Tipo: object.

Uso principal: lectura de áreas activadas.

natal_angles

Función: Ascendente, Descendente, Medio Cielo e IC.

Tipo: object.

Uso principal: activaciones fuertes por tránsito y validación estructural.

natal_aspects

Función: aspectos principales de la carta natal.

Tipo: array.

Uso principal: condición natal, resonancia con tránsitos, complejidad estructural.

natal_planet_condition

Función: estado interpretativo resumido de cada planeta natal.

Tipo: object.

Uso principal: Señor del Año, activaciones, matiz terapéutico.

Ejemplo:

```
{
  "Mars": {
    "strength": "medium",
    "tone": "impulsive_but_direct",
    "stability": "variable"
  }
}
```

natal_element_balance

Función: balance de elementos en la carta natal.

Tipo: object.

Uso principal: matrix, nivel de energía, apoyo interpretativo.

natal_modality_balance

Función: balance de modalidades en la carta natal.

Tipo: object.

Uso principal: matrix, ritmo de acción, tono estructural.

natal_dominant_planets

Función: planetas más relevantes de la carta natal según criterio interno del sistema.

Tipo: array.

Uso principal: personalización avanzada y priorización contextual.

natal_sensitive_points

Función: puntos sensibles disponibles si el sistema los utiliza.

Tipo: object.

Ejemplos: Nodos, Quirón, Lilith, Fortuna.

Uso principal: modos avanzados o futuras versiones.

8. BLOQUE C — VARIABLES DEL CICLO ANUAL

8.1. Función del bloque

Este bloque define el marco estructural del año actual del usuario. Es una de las capas más importantes del sistema.

Estas variables tienen prioridad alta porque sostienen la lectura global del ciclo.

8.2. Variables recomendadas

current_age

Función: edad actual del usuario en el ciclo vigente.

Tipo: number.

Origen: cálculo desde **birth_date**.

Obligatoria: sí.

Uso principal: profección anual.

current_profecion_year_start

current_profecion_year_end

Función: rango temporal del año profectado activo.

Tipo: date.

Uso principal: control temporal y trazabilidad.

annual_profecion_house

Función: casa activada por profección en el año vigente.

Tipo: number.

Obligatoria: sí.

Uso principal: estructura anual, prioridad interpretativa.

annual_profection_sign

Función: signo asociado a la casa profectada o a la activación anual.

Tipo: string.

Uso principal: matiz del ciclo anual.

lord_of_year

Función: planeta regente del año.

Tipo: string.

Obligatoria: sí.

Uso principal: eje central de la lectura anual y modulador del resto de vistas.

lord_of_year_natal_sign

Función: signo natal del Señor del Año.

Tipo: string.

Uso principal: matiz cualitativo.

lord_of_year_natal_house

Función: casa natal del Señor del Año.

Tipo: number.

Uso principal: área vital base que colorea el año.

lord_of_year_natal_condition

Función: síntesis del estado natal del Señor del Año.

Tipo: object o string resumida.

Uso principal: calidad de la experiencia anual.

lord_of_year_current_status

Función: estado actual del Señor del Año en tránsito o activación.

Tipo: object.

Uso principal: modular intensidad del presente según estructura anual.

annual_theme_primary

Función: tema dominante del año, ya derivado por el sistema.

Tipo: string.

Uso principal: anual, mensual, coherencia entre vistas.

annual_theme_secondary

Función: tema secundario del año.

Tipo: string.

Uso principal: matiz estructural.

annual_priority_score

Función: peso interpretativo del eje anual en la síntesis.

Tipo: number.

Uso principal: motor de jerarquía.

9. BLOQUE D — VARIABLES TEMPORALES DINÁMICAS

9.1. Función del bloque

Estas variables describen el estado temporal del momento consultado. No son estructura fija, sino clima activo.

Aquí se incluyen día, semana, mes, Luna actual, fase lunar, planeta del día y otros marcadores temporales.

9.2. Variables diarias

current_datetime

Función: fecha y hora exacta de la consulta o del render.

Tipo: datetime.

Obligatoria: sí.

current_date

Función: fecha activa simplificada.

Tipo: date.

Uso principal: diaria, semanal, mensual.

weekday

Función: día de la semana.

Tipo: string o number.

Uso principal: cálculo del planeta del día.

day_ruler

Función: planeta regente del día.

Tipo: string.

Obligatoria: sí.

Uso principal: clima diario, bloques cortos, cruces relacionales.

day_ruler_translated

Función: nombre del planeta del día en idioma de la app.

Tipo: string.

Uso principal: salida humana.

moon_sign_today

Función: signo actual de la Luna.

Tipo: string.

Obligatoria: sí.

Uso principal: clima emocional del día.

moon_house_today

Función: casa actual de la Luna respecto a la carta del usuario.

Tipo: number.

Uso principal: foco vital corto.

moon_phase_today

Función: fase lunar actual.

Tipo: string.

Obligatoria: sí.

Uso principal: ritmo de despliegue, cierre, tensión o siembra.

moon_phase_percentage

Función: porcentaje de iluminación lunar.

Tipo: number.

Uso principal: apoyo técnico y futuras escalas de intensidad.

solar_sign_current

Función: signo solar actual del calendario.

Tipo: string.

Uso principal: tono estacional o mensual.

daily_temporal_tone

Función: clima diario resumido ya derivado.

Tipo: string.

Uso principal: síntesis y módulos breves.

9.3. Variables semanales

week_start

week_end

Función: inicio y fin del período semanal activo.

Tipo: date.

weekly_focus

Función: foco temático principal de la semana.

Tipo: string.

Uso principal: semanal.

weekly_energy_pattern

Función: patrón energético resumido de la semana.

Tipo: string.

Uso principal: radar, weekly engine.

weekly_transition_type

Función: tipo de movimiento semanal.

Valores sugeridos: *expansion*, *consolidation*, *review*, *tension*, *integration*.

Tipo: string.

9.4. Variables mensuales

month_start

month_end

Función: inicio y fin del período mensual activo.

Tipo: date.

monthly_focus

Función: foco dominante del mes.

Tipo: string.

monthly_cycle_tone

Función: tono general del mes.

Tipo: string.

lunation_context

Función: lunación principal activa en el período.

Tipo: object o string.

Uso principal: integración mensual.

10. BLOQUE E — VARIABLES DE TRÁNSITOS Y ACTIVACIONES

10.1. Función del bloque

Este bloque recoge los tránsitos y activaciones relevantes del momento. Es uno de los bloques más delicados, porque si se desordena, KAIROS puede sobrereaccionar, dramatizar o perder jerarquía.

No todo tránsito merece entrar en la interpretación. Por eso este bloque debe distinguir entre:

- tránsitos detectados
- tránsitos relevantes
- tránsitos priorizados
- tránsitos descartados

10.2. Variables principales

detected_transits

Función: lista completa de tránsitos detectados por el sistema.

Tipo: array.

Uso principal: capa técnica previa.

relevant_transits

Función: tránsitos que superan los filtros mínimos de relevancia.

Tipo: array.

Uso principal: motor interpretativo.

prioritized_transits

Función: tránsitos ordenados por peso interpretativo.

Tipo: array.

Uso principal: síntesis real.

major_transits

Función: subconjunto de tránsitos de peso alto.

Tipo: array.

Uso principal: anual, mensual, semanal.

supportive_transits

Función: tránsitos con sesgo de apoyo, expansión o fluidez.

Tipo: array.

challenging_transits

Función: tránsitos con sesgo de tensión, fricción o exigencia.

Tipo: array.

transit_conflict_level

Función: nivel global de complejidad o ambivalencia del cielo activo.

Tipo: string o number.

Uso principal: conflicto, QA, longitud.

transit_signal_density

Función: cantidad de señal interpretativa presente en el período.

Tipo: number.

Uso principal: decidir fallback o condensación.

10.3. Estructura recomendada de cada tránsito

Cada tránsito importante debería modelarse como un objeto homogéneo.

```
{
  "transit_id": "saturn_square_moon_2026_03_12",
  "transit_planet": "Saturn",
  "natal_target": "Moon",
  "aspect": "square",
  "orb": 1.2,
  "applying": true,
  "exact_date": "2026-03-14",
```

```
"house_triggered": 4,  
"links_to_lord_of_year": true,  
"links_to_profection_house": false,  
"priority_score": 82,  
"tone": "challenging",  
"domains": ["emotion", "security", "responsibility"],  
"usable_in": ["weekly", "monthly"],  
"qa_flags": []  
}
```

10.4. Campos recomendados para cada tránsito

- transit_id
- transit_planet
- natal_target
- aspect
- orb
- applying
- separating
- exact_date
- start_date
- end_date
- house_triggered
- sign_context
- links_to_lord_of_year
- links_to_profection_house
- links_to_angle
- priority_score
- tone
- domains
- structural_weight
- usable_in
- status

10.5. Variables de activación complementaria

active_lunations

Función: lunaciones relevantes activas en el período.

Tipo: array.

active_retrogrades

Función: retrogradaciones activas relevantes.

Tipo: array.

active_house_triggers

Función: casas natales activadas por tránsitos o ciclo.

Tipo: array.

active_angle_triggers

Función: activación de ángulos importantes.

Tipo: array.

11. BLOQUE F — VARIABLES DERIVADAS POR EL MOTOR INTERPRETATIVO

11.1. Función del bloque

Aquí ocurre la alquimia real del sistema. Estas variables no vienen directamente del cálculo astronómico. Las produce KAIROS al integrar datos, jerarquía y biblioteca.

Son variables semánticas intermedias. Sin ellas, la salida final se dispersa o se vuelve mecánica.

11.2. Variables nucleares

dominant_theme

Función: tema principal del momento según la jerarquía activa.

Tipo: string.

Uso principal: síntesis central.

secondary_themes

Función: temas secundarios que acompañan al dominante sin desplazarlo.

Tipo: array.

active_life_area

Función: área vital más activada en el período.

Tipo: string.

Uso principal: enfoque de lectura.

emotional_climate

Función: clima emocional general del momento.

Tipo: string.

mental_climate

Función: clima mental o cognitivo dominante.

Tipo: string.

body_state_tone

Función: tono corporal o nivel de exigencia somática.

Tipo: string.

Uso principal: matrix y orientación práctica.

current_intensity_level

Función: nivel de intensidad global del período.

Tipo: string o number.

Valores sugeridos: low, medium, high, peak.

guidance_mode

Función: tipo de orientación principal que necesita la salida.

Tipo: string.

Valores sugeridos: contain, clarify, act, slow_down, integrate, observe.

warning_required

Función: indica si debe generarse advertencia principal.

Tipo: boolean.

warning_axis

Función: eje de la advertencia principal.

Tipo: string.

recommended_action_axis

Función: eje principal de acción recomendada.

Tipo: string.

reflection_axis

Función: pregunta o eje reflexivo central.

Tipo: string.

body_mind_spirit_balance

Función: estado resumido de la matrix interna.

Tipo: object.

Ejemplo:

```
{  
  "body": "overloaded",  
  "mind": "focused_but_rigid",  
  "spirit": "seeking_meaning"  
}
```

semantic_core

Función: núcleo interpretativo resumido del momento.

Tipo: object.

Uso principal: alimentar módulos de salida.

Estructura sugerida:

```
{  
  "theme": "reorden emocional con exigencia de madurez",  
  "focus": "vínculos y límites",  
  "tone": "intenso pero clarificador",  
  "risk": "reaccionar desde saturación",  
  "action": "poner orden antes de decidir",  
  "reflection": "qué estás sosteniendo por miedo a perder estabilidad"  
}
```

fallback_mode

Función: indica si el sistema ha tenido que usar un modo reducido por baja señal o falta de datos.

Tipo: boolean.

confidence_score

Función: nivel global de confianza interpretativa.

Tipo: number.

Uso principal: QA y control de lectura.

12. BLOQUE G — VARIABLES DE SALIDA

12.1. Función del bloque

Estas variables no describen el cielo ni el proceso del usuario. Describen **cómo va a empaquetarse la respuesta**.

Sirven para modular longitud, formato, profundidad y bloques activos.

12.2. Variables principales

output_type

Función: tipo de lectura solicitada.

Tipo: string.

Valores sugeridos: `annual`, `monthly`, `weekly`, `daily`, `matrix`, `journal`, `notification`.

output_modules

Función: módulos que deben renderizarse en esa salida.

Tipo: array.

Ejemplo: `["synthesis", "warning", "action", "reflection"]`.

output_length

Función: tamaño general de la respuesta.

Tipo: string.

Valores sugeridos: `micro`, `short`, `medium`, `long`.

tone_mode

Función: modo de tono narrativo.

Tipo: string.

Valores sugeridos: `therapeutic`, `clear`, `reflective`, `practical`, `contained`.

clarity_level

Función: nivel de simplificación del lenguaje.

Tipo: string.

Valores sugeridos: `high`, `normal`, `expert`.

`personalization_depth`

Función: profundidad de personalización permitida para esa salida.

Tipo: string.

Valores sugeridos: `basic`, `medium`, `deep`.

`cta_enabled`

Función: indica si hay llamada a la acción interna o invitación a explorar otro módulo.

Tipo: boolean.

`journal_prompt_enabled`

Función: indica si debe generarse prompt de diario.

Tipo: boolean.

`notification_ready`

Función: indica si la lectura puede condensarse a formato notificación.

Tipo: boolean.

13. BLOQUE H — VARIABLES DE CONTROL, TRAZABILIDAD Y QA

13.1. Función del bloque

Este bloque garantiza que KAIROS pueda revisar lo que hizo, con qué versión lo hizo y si la salida pasó o no criterios mínimos de calidad.

Es un bloque silencioso, pero clave para que el sistema no se vuelva opaco.

13.2. Variables recomendadas

`library_version`

Función: versión activa de la biblioteca consultada.

Tipo: string.

engine_version

Función: versión del motor interpretativo.

Tipo: string.

pipeline_version

Función: versión del flujo operativo.

Tipo: string.

qa_status

Función: estado general del control de calidad.

Tipo: string.

Valores sugeridos: `pending`, `passed`, `warning`, `failed`.

qa_flags

Función: lista de incidencias detectadas.

Tipo: array.

missing_data_flags

Función: datos ausentes o incompletos.

Tipo: array.

conflict_flags

Función: tensiones internas detectadas por el sistema.

Tipo: array.

cross_view_consistency_status

Función: estado de coherencia con vistas superiores o paralelas.

Tipo: string.

generated_at

Función: momento exacto de generación de la lectura.

Tipo: datetime.

source_nodes_used

Función: lista de nodos documentales utilizados para construir la salida.

Tipo: array.

Uso principal: auditoría y trazabilidad.

computation_trace_id

Función: identificador de la ejecución concreta del motor.

Tipo: string.

14. CLASIFICACIÓN TRANSVERSAL DE VARIABLES

Además del bloque al que pertenecen, cada variable importante debe clasificarse según varios ejes.

14.1. Clasificación por origen

source: user_input

Variables introducidas por el usuario.

Ejemplo: `birth_date`, `birth_time`, `birth_place_name`.

source: computed

Variables calculadas por motores astronómicos o lógicos.

Ejemplo: `annual_profection_house`, `day_ruler`, `relevant_transits`.

source: derived

Variables construidas por el motor interpretativo.

Ejemplo: `dominant_theme`, `guidance_mode`, `semantic_core`.

source: system

Variables técnicas de control.

Ejemplo: `engine_version`, `qa_status`, `generated_at`.

14.2. Clasificación por prioridad interpretativa

priority_level: structural

Variables que sostienen el sentido del ciclo.

Ejemplos:

- `annual_profection_house`
- `lord_of_year`
- `lord_of_year_natal_condition`

priority_level: dynamic

Variables que activan o intensifican el presente.

Ejemplos:

- `prioritized_transits`
- `active_lunations`
- `monthly_focus`

priority_level: modulatory

Variables que colorean el clima sin redefinir la estructura.

Ejemplos:

- `moon_sign_today`
- `day_ruler`
- `moon_phase_today`

priority_level: output

Variables que gobiernan la forma de entrega.

Ejemplos:

- `output_type`
- `output_modules`
- `output_length`

priority_level: control

Variables técnicas de QA y trazabilidad.

Ejemplos:

- `qa_status`
- `missing_data_flags`
- `library_version`

14.3. Clasificación por obligatoriedad

required: critical

Sin esta variable el sistema no debería generar lectura completa.

Ejemplo:

- `user_id`
- `birth_date`
- `timezone`
- `annual_profection_house`
- `lord_of_year`

required: recommended

La salida puede existir sin ella, pero pierde calidad.

Ejemplo:

- `birth_time`
- `natal_aspects`
- `natal_element_balance`

required: optional

Enriquece, pero no bloquea.

Ejemplo:

- `user_preferences`
- `notification_ready`

15. ESTRUCTURA RECOMENDADA DE REGISTRO DE VARIABLE

Para integración ordenada, cada variable relevante debería documentarse con esta plantilla:

Ficha de variable

- **nombre interno**
- **etiqueta humana**
- **bloque**
- **descripción funcional**
- **tipo de dato**
- **origen**
- **módulo que la calcula**
- **obligatoriedad**
- **frecuencia de actualización**

- prioridad interpretativa
- vistas donde se usa
- ejemplo
- notas de QA

Ejemplo

Variable: **lord_of_year**

- **etiqueta humana:** Señor del Año
 - **bloque:** ciclo anual
 - **descripción funcional:** planeta regente del año profectado activo del usuario
 - **tipo de dato:** string
 - **origen:** computed
 - **módulo que la calcula:** `projection_engine.js`
 - **obligatoriedad:** critical
 - **frecuencia de actualización:** anual
 - **prioridad interpretativa:** structural
 - **vistas donde se usa:** annual, monthly, weekly, daily
 - **ejemplo:** `Mars`
 - **notas de QA:** no debe ser desplazado por factores diarios menores
-

16. REGLAS GENERALES DE USO DE VARIABLES

16.1. No usar variables rápidas como si fueran estructura

La Luna del día, la fase lunar o el planeta del día modulan. No deben gobernar la lectura anual ni imponerse sobre el Señor del Año.

16.2. No usar variables derivadas sin trazabilidad

Toda variable derivada debe poder vincularse a los factores que la generaron.

16.3. No mezclar cálculo con narración

Una variable debe describir estado, no texto final.

Por ejemplo:

- correcto: `dominant_theme = "reorden emocional"`
- incorrecto: `daily_text = "Hoy será un día..."`

16.4. No duplicar variables con nombres distintos

Si ya existe `moon_sign_today`, no crear otra variable paralela con otro nombre para la misma función.

16.5. Toda variable relevante debe poder auditarse

El sistema debe poder saber qué cálculo, qué nodo o qué regla alimentó una salida concreta.

17. RELACIÓN DEL ESQUEMA DE VARIABLES CON LA ARQUITECTURA TÉCNICA

17.1. Relación con motores JS

Este documento debe mapearse con los motores ya existentes o previstos:

- `chart_engine.js` → variables natales
- `ascendant_engine.js` → ascendente y casas
- `astronomy.js / swissep` → cálculos astronómicos base
- `projection_engine.js` → profección y Señor del Año
- `transit_engine.js` → tránsitos y activaciones
- `daily_engine.js` → armado de contexto diario
- `weekly_engine.js` → armado de contexto semanal
- `monthly_engine.js` → armado de contexto mensual
- `radar_engine.js` → energía e intensidad
- `history_engine.js` → persistencia y trazabilidad temporal

17.2. Relación con Firestore

La app puede guardar variables en distintos niveles:

Perfil usuario

- datos de identidad
- carta natal persistente
- preferencias

Estado astral calculado

- contexto anual
- contexto mensual
- contexto diario
- tránsitos priorizados

Historial de lecturas

- semantic core generado
- módulos entregados
- QA
- timestamp
- versión usada

17.3. Relación con biblioteca estructurada

Las variables no deben guardar el texto final completo de la biblioteca. Deben servir para consultar nodos interpretativos adecuados.

Ejemplo:

- `lord_of_year = Venus`
- `annual_profection_house = 7`
- `moon_sign_today = Aries`

Estas variables permiten consultar:

- nodo “Señor del Año Venus”
- nodo “Casa 7 activada por profección”
- nodo “Luna en Aries”
- nodo de cruce si procede

18. Criterio de mantenimiento

Este esquema debe actualizarse cuando ocurra cualquiera de estas situaciones:

- se incorpore una nueva vista de salida
- se añada un nuevo factor astrológico al sistema
- cambie la arquitectura de almacenamiento

- cambie la lógica de cálculo de una variable
 - se detecte duplicidad o ambigüedad en nombres
 - se añadan modos avanzados
 - el motor de QA exija nuevos metadatos
-

19. Cierre del documento

El esquema de variables de KAIROS no es un anexo técnico secundario. Es una pieza central del sistema.

La biblioteca puede ser profunda.

Los motores astronómicos pueden ser precisos.

La salida puede sonar humana.

Pero si las variables están mal definidas, todo eso se rompe por debajo.

Este documento existe para impedirlo.

Su función es asegurar que KAIROS no trabaje con intuiciones sueltas, sino con una estructura limpia donde cada dato tenga:

- un nombre claro
- un origen claro
- una función clara
- una prioridad clara
- y una trazabilidad real

Solo así la app puede interpretar con orden en lugar de reaccionar con ruido.

003_MOTOR_DE_ORQUESTACION_INTERPRETATIVA_DE_KAIROS

Sistema KAIROS

Definición del cerebro lógico que integra variables, jerarquía, biblioteca y salida humana

1. Propósito del documento

Este documento define el **motor de orquestación interpretativa de KAIROS**. Su función es establecer cómo el sistema recibe datos astrológicos reales, identifica los factores relevantes del ciclo, aplica jerarquía interpretativa, consulta la biblioteca adecuada, resuelve tensiones y construye una respuesta coherente, clara y terapéutica para el usuario.

KAIROS no debe operar como un horóscopo automático ni como un generador de frases astrológicas. Debe funcionar como un **sistema experto de orientación interpretativa**, capaz de distinguir entre:

- estructura y clima
- contexto y activación
- intensidad y ruido
- señal principal y señal secundaria
- significado y forma de expresión

Este documento no define textos concretos de salida, sino la lógica que permite producirlos sin contradicción, sin arbitrariedad y sin sobre-reacción ante factores menores.

2. Función dentro de KAIROS

El motor de orquestación interpretativa cumple seis funciones centrales:

1. Recibir el estado astrológico del usuario

Integra datos natales, ciclo anual, contexto temporal y activaciones del momento.

2. Distinguir qué factores pesan realmente

No todo lo calculado debe entrar en la interpretación final.

3. Aplicar jerarquía

Determina qué factor sostiene el sentido del momento y cuáles solo lo modulan.

4. Consultar la biblioteca relevante

Selecciona únicamente los nodos documentales útiles para la vista solicitada.

5. Generar un núcleo interpretativo coherente

Produce una síntesis semántica intermedia antes de renderizar módulos humanos.

6. Gobernar la salida final

Entrega a los motores de salida un material ya ordenado, priorizado y validado.

3. Alcance

Este documento incluye:

- principios del motor
- estructura lógica de funcionamiento

- capas de jerarquía interpretativa
- criterios de activación
- scoring orientativo
- resolución de conflicto
- reglas de fallback
- construcción del núcleo semántico
- relación con módulos de salida

Este documento no sustituye:

- al esquema de variables
- al pipeline técnico
- a las reglas de QA
- a la biblioteca semántica
- a la biblioteca narrativa

Su función es definir **cómo se toma la decisión interpretativa**.

4. Principio rector del motor

El motor de KAIROS debe obedecer una regla central:

la interpretación no se construye por acumulación de factores, sino por jerarquía de sentido.

Esto significa:

- no gana quien más ruido haga
- no entra todo lo calculado
- no se suman significados como si fueran piezas de Lego sin criterio
- no se permite que un factor rápido desplace la estructura del ciclo
- no se permite que la salida final suene intensa si la señal real no lo es

KAIROS debe interpretar como un sistema con criterio, no como una trituradora de símbolos.

5. Naturaleza del motor

El motor interpretativo de KAIROS debe ser **principalmente determinista**.

Eso implica:

- la lógica principal no depende de un LLM
- las prioridades se deciden por reglas del sistema
- los factores se seleccionan por criterios explícitos
- la resolución de conflicto sigue reglas internas
- la salida final se construye desde nodos estructurados y módulos de ensamblaje

Un modelo generativo, si se usa más adelante, solo debería intervenir como:

- capa de pulido estilístico
- ajuste fino de naturalidad
- condensación lingüística controlada

Nunca como cerebro principal.

6. Principios operativos del motor

6.1. Primacía de la estructura anual

La profección anual, la casa activada y el Señor del Año constituyen el marco estructural del ciclo. Ningún factor rápido debe anular ese eje.

6.2. Los tránsitos activan, no sustituyen

Los tránsitos relevantes intensifican, tensan, precipitan o facilitan procesos, pero deben leerse en relación con la estructura anual.

6.3. El clima corto modula

Luna actual, fase lunar y planeta del día sirven para modular el tono del momento, no para redefinir el sentido profundo del ciclo.

6.4. La interpretación debe condensar

El sistema debe reducir complejidad, no trasladarla al usuario sin filtrar.

6.5. Toda salida debe surgir de un núcleo semántico

Antes de redactar síntesis, advertencias o acciones, el motor debe haber producido una estructura interna coherente de sentido.

6.6. El motor debe saber callar

Si la señal es débil, la salida debe ser más ligera. Si faltan datos, debe activarse fallback. Si no hay claridad suficiente, no debe inventarse intensidad.

7. Posición del motor dentro de la arquitectura de KAIROS

El motor de orquestación se sitúa entre:

capa de entrada y cálculo

- carta natal
- tránsitos
- profección
- Señor del Año
- Luna actual
- fase lunar
- planeta del día

y

capa de salida humana

- síntesis
- advertencia
- acción recomendada
- pregunta de reflexión
- lectura diaria
- lectura semanal
- lectura mensual
- lectura anual
- matrix
- diario personal

Por tanto, su función es **transformar cálculo en sentido organizado**.

8. Entradas del motor

El motor debe recibir un objeto de estado normalizado. No debe depender de datos dispersos o de módulos que se llamen entre sí de forma caótica.

8.1. Estructura general de entrada recomendada

```
{
  "user_context": {},
  "natal_context": {},
  "annual_context": {},
  "temporal_context": {},
  "transit_context": {},
  "output_context": {},
  "control_context": {}
}
```

}

8.2. Bloques mínimos de entrada

user_context

Idioma, zona horaria, nivel de acceso, configuración de salida.

natal_context

Carta natal, casas, planetas, ángulos, condición natal básica.

annual_context

Edad actual, profección anual, casa activada, Señor del Año, estado natal del Señor del Año.

temporal_context

Fecha activa, día de la semana, planeta del día, Luna actual, fase lunar, Sol actual, período semanal o mensual.

transit_context

Tránsitos detectados, tránsitos relevantes, tránsitos priorizados, lunaciones activas, activaciones por casa o ángulos.

output_context

Tipo de lectura solicitada, módulos deseados, longitud, tono, profundidad.

control_context

Versiones, trazabilidad, flags de datos incompletos, nivel de confianza.

9. Capas interpretativas del motor

KAIROS debe razonar por capas. Esta es la lógica central del motor.

9.1. Capa 1 — Estructura del ciclo

Esta capa define el marco profundo del período actual.

Factores principales:

- casa activada por profección
- Señor del Año
- condición natal del Señor del Año
- relación entre Señor del Año y casa activada
- grandes activaciones del año

Función:

Establecer el eje de sentido dominante del ciclo.

Pregunta que responde:

¿De qué va este período en su nivel estructural?

9.2. Capa 2 — Activación del presente

Esta capa determina qué procesos están siendo movilizados con fuerza en el momento actual.

Factores principales:

- tránsitos mayores priorizados
- activación de casas sensibles
- contacto con ángulos
- resonancia con Señor del Año
- lunaciones relevantes
- foco mensual

Función:

Traducir la estructura anual a una activación concreta y viva.

Pregunta que responde:

¿Qué está moviéndose ahora con suficiente fuerza como para entrar en la lectura?

9.3. Capa 3 — Clima temporal corto

Esta capa modula el tono del día o de la semana.

Factores principales:

- Luna en tránsito por signo y casa
- fase lunar
- planeta del día

- ritmo solar estacional
- microclima emocional del momento

Función:

Colorear la experiencia inmediata sin desplazar el eje del ciclo.

Pregunta que responde:

¿Cómo se siente o se expresa hoy este proceso?

9.4. Capa 4 — Traducción humana

Esta capa no calcula ni decide significado estructural. Traduce el resultado a módulos comprensibles para el usuario.

Factores principales:

- síntesis breve
- advertencia
- acción recomendada
- pregunta de reflexión
- matrix
- diario

Función:

Convertir el núcleo interpretativo en experiencia legible y útil.

Pregunta que responde:

¿Cómo se expresa esto de forma clara, terapéutica y accionable?

10. Regla jerárquica central

El motor debe aplicar siempre esta jerarquía:

Prioridad 1 — Estructura anual

- profección anual
- casa activada
- Señor del Año
- condición natal del Señor del Año

Prioridad 2 — Activadores fuertes

- tránsitos mayores con peso real

- activaciones de la casa proyectada
- aspectos al Señor del Año
- contactos a ángulos o planetas personales relevantes

Prioridad 3 — Ritmo de período

- lunaciones
- foco mensual
- patrones semanales

Prioridad 4 — Modulación corta

- Luna del día
- fase lunar
- planeta del día

Regla obligatoria

Los factores rápidos modulan; no redefinen.

Esa frase debería quedar como norma explícita del sistema.

11. Modo de trabajo del motor

El motor interpretativo debe operar en ocho fases.

FASE 1 — Validación de entrada

Objetivo

Comprobar que existen datos mínimos suficientes para generar una lectura válida.

Verificaciones básicas

- carta natal disponible
- fecha activa válida
- zona horaria válida
- profección anual calculable
- Señor del Año disponible
- contexto de salida definido

Resultado esperado

- `input_valid = true/false`

- `missing_data_flags`
 - decisión de continuar o activar fallback
-

FASE 2 — Construcción del estado interpretativo

Objetivo

Unificar en un solo objeto el contexto completo del usuario y del momento.

Acción del motor

Integrar:

- datos natales
- datos anuales
- datos del período
- tránsitos
- preferencias de salida

Resultado esperado

Un objeto estable sobre el que ya puede trabajar la lógica.

FASE 3 — Detección de factores activos

Objetivo

Determinar qué factores del sistema están realmente activos y cuáles no merecen entrar todavía en la lectura.

Regla

No todo lo detectado entra.

Filtros posibles

- orbe demasiado amplia
- tránsito irrelevante para esa vista
- activación sin resonancia estructural
- señal redundante ya cubierta por otro factor mayor
- factor menor sin suficiente intensidad

Resultado esperado

- `active_structural_factors`
 - `active_dynamic_factors`
 - `active_modulatory_factors`
 - `discarded_factors`
-

FASE 4 — Asignación de peso interpretativo

Objetivo

Asignar prioridad a los factores seleccionados.

11.4.1. Scoring orientativo recomendado

Los números no son verdad metafísica. Son una herramienta de orden.

Estructura anual

- casa proyectada activa: 100
- Señor del Año: 95
- condición natal del Señor del Año: 90

Activadores fuertes

- tránsito mayor exacto al Señor del Año: 90
- tránsito mayor a planeta personal relevante: 85
- activación de ángulo: 82
- activación de casa proyectada: 80
- lunación relevante conectada al eje anual: 75

Ritmo mensual/semanal

- foco mensual activo: 68
- patrón semanal fuerte: 60

Clima corto

- fase lunar: 50
- Luna por signo/casa: 40
- planeta del día: 25

Regla de uso

El scoring no sustituye la lógica, pero la ayuda.

Sirve para que el motor no se pierda ni se deje arrastrar por un detalle brillante pero secundario.

Resultado esperado

Lista priorizada de factores con peso.

FASE 5 — Resolución de conflicto

Objetivo

Integrar factores que pueden parecer contradictorios sin romper la coherencia de la lectura.

Principios de resolución

a) Gana el factor más estructural

Si hay conflicto entre un tránsito rápido y el eje anual, el tránsito se interpreta como matiz, no como redefinición.

b) El segundo factor no desaparece

Se integra como tensión, ambivalencia o contraste, no se borra.

c) Dos factores fuertes no se anulan: se articulan

Si hay expansión y exigencia al mismo tiempo, el sistema debe expresarlo como crecimiento con coste, apertura con responsabilidad o impulso con necesidad de orden.

d) El conflicto debe traducirse, no exhibirse en bruto

El usuario no necesita ver contradicción caótica. Necesita ver sentido complejo pero comprensible.

Resultado esperado

- resolved_theme
 - resolved_tension_axis
 - dominant_factor
 - secondary_factors
-

FASE 6 — Selección de nodos de biblioteca

Objetivo

Consultar solo la biblioteca necesaria para construir la salida.

Regla

El motor no debe leer “todo”. Debe consultar de forma precisa.

Ejemplo de lectura diaria

Podría consultar:

- nodo de fase lunar actual
- nodo de planeta del día
- nodo de Luna en signo
- nodo de cruce Luna + planeta del día si aplica
- nodo del Señor del Año si hay resonancia
- reglas de prioridad
- reglas de fallback
- plantillas de síntesis diaria

Ejemplo de lectura mensual

Podría consultar:

- nodo de Señor del Año por planeta
- nodo de casa activada
- nodos de tránsitos mayores priorizados
- nodo de signo solar actual
- nodos de lunación relevante
- reglas de conflicto
- reglas de coherencia entre vistas
- módulos narrativos mensuales

Resultado esperado

- `selected_semantic_nodes`
 - `selected_relational_nodes`
 - `selected_rules`
 - `selected_output_templates`
-

FASE 7 — Construcción del núcleo semántico

Objetivo

Producir una estructura de sentido intermedia antes de redactar módulos.

Esta es la fase más importante.

Sin núcleo semántico, la salida se vuelve:

- repetitiva
- mecánica
- fragmentada
- contradictoria
- demasiado literal

11.7.1. Estructura mínima recomendada del núcleo semántico

```
{  
  "dominant_theme": "",  
  "secondary_themes": [],  
  "life_area_focus": "",  
  "energy_tone": "",  
  "emotional_climate": "",  
  "main_risk": "",  
  "main_opportunity": "",  
  "recommended_action_axis": "",  
  "reflection_axis": "",  
  "guidance_mode": "",  
  "intensity_level": "",  
  "warning_required": false  
}
```

11.7.2. Función de cada campo

dominant_theme

La idea central que sostiene la lectura.

secondary_themes

Matizes importantes que no deben competir con el eje principal.

life_area_focus

Área vital más movilizada.

energy_tone

Calidad general del momento: denso, abierto, exigente, fértil, inestable, clarificador, etc.

emotional_climate

Traducción del tono afectivo predominante.

main_risk

Dónde puede torcerse el proceso si no se regula bien.

main_opportunity

Qué posibilidad de crecimiento, orden o comprensión ofrece el momento.

recommended_action_axis

Qué dirección práctica conviene tomar.

reflection_axis

Qué pregunta o eje interior merece observación.

guidance_mode

Modo de acompañamiento dominante:

- contener
- aclarar
- actuar
- ordenar
- bajar velocidad
- integrar

- observar

intensity_level

Nivel de intensidad global.

warning_required

Si procede generar advertencia principal.

FASE 8 — Mapeo a módulos de salida

Objetivo

Entregar a los motores de salida material ya organizado.

Regla

Los módulos de salida no deben reinterpretar. Deben renderizar.

Ejemplos de consumo por modo

daily_mode

Consume:

- dominant_theme
- energy_tone
- main_risk
- recommended_action_axis
- reflection_axis

weekly_mode

Consume:

- dominant_theme
- secondary_themes
- life_area_focus
- emotional_climate
- main_risk
- main_opportunity
- action_axis
- reflection_axis

monthly_mode

Consume:

- eje anual aplicado al mes
- life area focus
- dominant theme
- secondary themes
- dynamic triggers
- risk + opportunity
- maduración del período

annual_mode

Consume:

- estructura anual completa
- casa activada
- Señor del Año
- condición natal del Señor del Año
- narrativa de desarrollo del ciclo

matrix_mode

Consume:

- body_state_tone
- mental_climate
- emotional_climate
- body_mind_spirit_balance
- current_intensity_level

journal_mode

Consume:

- reflection_axis
- emotional_climate
- dominant_theme

Resultado esperado

Paquete de módulos listos para la capa de salida.

12. Modos operativos del motor

El motor no debe pensar igual en todas las vistas. Debe activar un modo según la lectura solicitada.

12.1. **annual_mode**

Función

Construir la narrativa estructural del año.

Factores dominantes

- profección
- Señor del Año
- condición natal del regente anual
- activaciones fuertes del año

Rasgo principal

Alta profundidad, baja dependencia de clima corto.

12.2. **monthly_mode**

Función

Traducir el mes dentro del año activo.

Factores dominantes

- eje anual
- activaciones del mes
- lunaciones
- foco solar
- tránsitos priorizados

Rasgo principal

Puente entre estructura anual y experiencia concreta.

12.3. **weekly_mode**

Función

Detectar patrón, contraste y enfoque de la semana.

Factores dominantes

- continuidad mensual
- tránsitos útiles
- clima lunar
- cambios de ritmo

Rasgo principal

Lectura media, enfocada y no excesivamente densa.

12.4. **daily_mode**

Función

Ofrecer clima breve y útil del día.

Factores dominantes

- Luna actual
- fase lunar
- planeta del día
- resonancia con eje anual
- tránsito fuerte solo si realmente pesa

Rasgo principal

Brevedad, claridad, contención.

12.5. **matrix_mode**

Función

Traducir el estado del momento a los ejes cuerpo, mente y espíritu.

Rasgo principal

No añade interpretación paralela; reorganiza la ya existente.

12.6. **journal_mode**

Función

Convertir el núcleo interpretativo en reflexión íntima guiada.

Rasgo principal

Profundidad breve, no repetición, tono recogido.

12.7. **notification_mode**

Función

Condensar el núcleo interpretativo en una frase muy breve o aviso útil.

Rasgo principal

Minimalismo sin banalización.

13. **Criterios de activación real**

El motor debe decidir qué entra y qué no según criterios explícitos.

13.1. **Un factor entra con facilidad si:**

- activa la casa proyectada
- toca al Señor del Año
- toca un planeta personal relevante
- toca un ángulo
- tiene orbe estrecha
- coincide con una lunación significativa
- refuerza un tema ya presente en la estructura anual

13.2. **Un factor entra con prudencia si:**

- es rápido
- es moduladorio
- su señal es ambigua
- aporta color, pero no dirección

13.3. **Un factor puede descartarse si:**

- no cambia nada sustancial
- duplica otro factor ya más fuerte
- tiene relevancia técnica pero no interpretativa
- introduce ruido más que sentido
- no corresponde a la vista solicitada

14. Regla de condensación

Aunque existan muchos factores activos, el motor no debe entregar una lectura inflada.

Límite recomendado:

- 1 tema dominante
- hasta 2 temas secundarios
- 1 riesgo principal
- 1 oportunidad principal
- 1 acción principal
- 1 pregunta principal

Esto protege la inteligibilidad.

15. Regla de fallback

El motor debe tener un modo de funcionamiento reducido cuando:

- faltan datos
- la señal es baja
- no hay activadores fuertes
- la vista pedida no justifica una lectura intensa

15.1. Fallback mínimo recomendado

Para lectura diaria:

- estructura anual básica
- Luna actual
- fase lunar
- planeta del día

Para lectura semanal:

- tono mensual
- tránsito más claro si lo hay
- clima lunar dominante

Para lectura mensual:

- eje anual
- una activación principal
- lunación relevante si existe

15.2. Regla narrativa de fallback

Cuando el sistema cae en fallback:

- no debe fingir profundidad épica
- debe ser útil, sobrio y claro
- debe reconocer implícitamente que el momento no exige dramatización

16. Regla de contención interpretativa

El motor debe impedir estas deformaciones:

16.1. Sobredramatización

No convertir un tránsito menor o una Luna intensa en destino estructural.

16.2. Exceso técnico

No trasladar al usuario la complejidad del cálculo como si eso fuera profundidad.

16.3. Fragmentación

No producir bloques que parecen escritos por cinco sistemas distintos.

16.4. Contradicción

No decir una cosa en síntesis y otra incompatible en acción o advertencia.

16.5. Inflación semántica

No usar palabras grandes cuando la señal es pequeña.

17. Relación del motor con la biblioteca documental

El motor no debe consultar PDFs completos en tiempo real. Debe trabajar con una versión estructurada de la biblioteca.

17.1. Tipos de nodos que debe consultar

Nodos semánticos

Significados base:

- planeta
- signo
- casa
- fase lunar
- planeta del día
- profección
- Señor del Año

Nodos relacionales

Cruces:

- planeta en signo
- planeta en casa
- casa activada
- Señor del Año por planeta
- Luna en signo
- cruces entre factores

Nodos de reglas

- prioridad
- conflicto
- fallback
- longitud
- coherencia
- personalización

Nodos narrativos

- síntesis
- advertencia
- acción
- reflexión
- diario
- matrix

17.2. Regla de consulta

La consulta debe ser:

- precisa
- mínima
- contextual
- trazable

No conviene que el motor cargue más material del necesario.

18. Relación del motor con la arquitectura técnica

18.1. Relación con motores JS existentes

El motor de orquestación debe recibir insumos de:

- `chart_engine.js`
- `ascendant_engine.js`
- `astronomy.js`
- `projection_engine.js`
- `transit_engine.js`
- `planetary_engine.js`

Y entregar resultados a:

- `daily_engine.js`
- `weekly_engine.js`
- `monthly_engine.js`
- `radar_engine.js`
- `history_engine.js`
- capa UI

18.2. Rol de un posible archivo central

Técnicamente convendría crear un orquestador central, algo del tipo:

- `kairos_orchestrator.js`

o equivalente, cuya función no sea renderizar HTML, sino:

- consolidar contexto
- correr scoring
- resolver jerarquía
- construir núcleo semántico
- llamar a módulos de salida

Eso evitaría que `engine_v483_final.js` acabe cargando demasiadas responsabilidades y se convierta en un pulpo con ansiedad.

19. Estructura funcional resumida del motor

1. validar entrada

2. construir estado interpretativo
3. detectar factores activos
4. asignar pesos
5. resolver conflicto
6. seleccionar nodos
7. construir núcleo semántico
8. mapear a módulos de salida
9. entregar a pipeline y QA

20. Indicadores de buen funcionamiento del motor

El motor está funcionando bien si consigue:

- respetar la estructura anual
- no sobrerreaccionar a factores menores
- producir una lectura clara aunque el cielo esté complejo
- mantener coherencia entre diaria, semanal, mensual y anual
- traducir ambivalencia sin contradicción
- activar fallback sin empobrecer demasiado la experiencia
- generar un núcleo semántico útil para varios módulos

21. Indicadores de mal funcionamiento del motor

Hay señales claras de que el motor está pensando mal si:

- la diaria contradice la mensual
- la Luna del día manda más que el Señor del Año
- la salida cambia de tono sin motivo
- aparecen demasiados temas a la vez
- la advertencia dramatiza más que la señal real
- la acción recomendada no deriva del núcleo interpretativo
- el sistema parece sumar frases en vez de organizar sentido

22. Criterio de mantenimiento

Este documento debe revisarse cuando ocurra cualquiera de estas situaciones:

- se añada una nueva familia de variables
- cambie la jerarquía interpretativa del sistema

- se incorporen nuevos factores astrológicos
- se creen nuevas vistas de salida
- cambie la arquitectura de biblioteca estructurada
- se detecten contradicciones recurrentes en QA
- se incorpore una capa generativa adicional

23. Cierre del documento

El motor de orquestación interpretativa es el verdadero corazón de KAIROS.

La carta natal aporta la estructura.

Los cálculos aportan hechos.

La biblioteca aporta significado.

La salida humana aporta lenguaje.

Pero nada de eso basta si no existe una inteligencia interna que sepa:

- qué mirar primero
- qué dejar fuera
- qué pesa más
- qué matiza
- qué tensión integrar
- y cómo traducirlo sin ruido

Ese es el trabajo de este motor.

KAIROS no debe limitarse a calcular el cielo ni a ensamblar frases bonitas.

Debe saber pensar interpretativamente con orden, jerarquía y humanidad.

Este documento existe para que ese pensamiento no dependa de intuiciones cambiantes, sino de una lógica explícita, trazable y mantenible.

004_PIPELINE_DE_GENERACION_DE_RESPUESTA

Sistema KAIROS

Flujo operativo desde la entrada de datos hasta la salida modular en la app

1. Propósito del documento

Este documento define el **pipeline de generación de respuesta de KAIROS**, es decir, el recorrido completo que sigue el sistema desde que recibe una solicitud hasta que entrega una lectura visible, modular, coherente y validada al usuario.

Su función es convertir la lógica del motor interpretativo en un proceso operativo claro, repetible y mantenible. Este pipeline debe servir como referencia común para:

- arquitectura del sistema
- implementación técnica en Antigravity
- integración con Firebase y Firestore
- organización de motores JS
- depuración de errores
- trazabilidad de salidas
- control de calidad final

KAIROS no debe generar respuestas de forma improvisada ni por ensamblaje difuso entre módulos sueltos. Debe seguir un flujo definido donde cada etapa tenga una función clara y un límite claro.

2. Función dentro de KAIROS

El pipeline de generación de respuesta cumple siete funciones centrales:

1. Ordenar la ejecución del sistema

Define en qué orden deben ocurrir los cálculos, consultas, filtros, síntesis y validaciones.

2. Unificar capas distintas

Conecta cálculo astronómico, variables del sistema, biblioteca estructurada, motor interpretativo, salida humana y QA.

3. Evitar lógica dispersa

Impide que cada motor genere respuestas por su cuenta sin pasar por un eje común de decisión.

4. Facilitar implementación técnica

Permite traducir la lógica del sistema a funciones, servicios, colecciones y módulos de frontend/backend.

5. Permitir trazabilidad

Hace posible saber qué datos, reglas y nodos alimentaron una respuesta concreta.

6. Preparar mantenimiento

Ayuda a modificar una fase del sistema sin romper las demás.

7. Delimitar responsabilidades

Aclara qué hace cada etapa y qué no debe hacer.

3. Alcance

Este documento incluye:

- fases del pipeline
- entradas y salidas de cada fase
- dependencias entre fases
- criterios de validación
- interacción con Firestore
- interacción con motores JS
- interacción con vistas de salida
- reglas de entrega modular

Este documento no sustituye:

- al esquema de variables
- al motor de orquestación
- a las reglas de QA
- a la biblioteca interpretativa

Su función es definir **cómo se ejecuta el sistema**, no qué significa cada factor ni cómo se redacta cada texto.

4. Principio general del pipeline

El pipeline de KAIROS debe obedecer una regla básica:

ninguna salida debe renderizarse antes de que el sistema haya construido, jerarquizado y validado un núcleo interpretativo.

Eso implica:

- primero se calculan y normalizan datos
- después se detectan factores activos
- después se aplica jerarquía
- después se selecciona biblioteca
- después se construye el núcleo semántico
- después se generan módulos de salida
- y solo al final se renderiza la interfaz

No al revés.

5. Visión general del pipeline

El flujo general recomendado para KAIROS es este:

1. entrada de solicitud

2. carga de contexto de usuario
3. carga o cálculo de contexto astral
4. normalización de variables
5. detección de factores activos
6. jerarquización y resolución interpretativa
7. selección de nodos de biblioteca
8. construcción del núcleo semántico
9. ensamblaje de módulos de salida
10. control de tono y longitud
11. QA automático
12. persistencia e historial
13. entrega a UI

6. Arquitectura lógica del pipeline

El pipeline puede organizarse en seis capas funcionales:

Capa 1 — Entrada

Recoge la solicitud y el contexto de usuario.

Capa 2 — Datos astrales

Carga o calcula el estado astrológico requerido.

Capa 3 — Orquestación interpretativa

Evalúa, jerarquiza y sintetiza el sentido del momento.

Capa 4 — Salida modular

Construye los bloques que verá el usuario.

Capa 5 — Control y QA

Verifica calidad, coherencia y seguridad narrativa.

Capa 6 — Persistencia y entrega

Guarda trazabilidad y renderiza en la app.

7. FASE 1 — ENTRADA DE SOLICITUD

7.1. Objetivo

Capturar de forma clara qué está pidiendo el sistema o el usuario en ese momento.

7.2. Entradas mínimas

El pipeline debe recibir al menos:

- `user_id`
- `output_type`
- `current_datetime`
- `lang`
- `timezone`

Opcionalmente:

- configuración de longitud
- módulos específicos solicitados
- modo de suscripción
- flags de debug
- modo experto o modo normal

7.3. Ejemplos de solicitudes válidas

Lectura diaria

```
{  
  
  "user_id": "abc123",  
  
  "output_type": "daily",  
  
  "lang": "es",  
  
  "timezone": "Europe/Madrid",  
  
  "current_datetime": "2026-03-12T10:30:00"
```

```
}
```

Lectura mensual

```
{  
  "user_id": "abc123",  
  "output_type": "monthly",  
  "lang": "es",  
  "timezone": "Europe/Madrid",  
  "current_datetime": "2026-03-12T10:30:00",  
  "output_length": "medium"  
}
```

7.4. Salida de la fase

La fase 1 debe producir un objeto `request_context` limpio y validado.

8. FASE 2 — CARGA DE CONTEXTO DE USUARIO

8.1. Objetivo

Recuperar los datos persistentes del usuario que el sistema necesita para construir el estado interpretativo.

8.2. Datos que deben recuperarse

Desde Firestore o almacenamiento equivalente:

- perfil de usuario
- idioma
- zona horaria

- fecha de nacimiento
- hora de nacimiento
- lugar de nacimiento
- carta natal persistida si ya existe
- preferencias de experiencia
- nivel de acceso
- historial reciente si procede

8.3. Reglas

a) Priorizar persistencia sobre recalcular lo estable

La carta natal no debe recalcularse desde cero en cada lectura si ya está validada y almacenada.

b) Validar consistencia de datos

Si falta hora natal o coordenadas, el pipeline debe detectarlo aquí.

c) Marcar precisión

Debe arrastrarse el nivel de fiabilidad de la hora natal.

8.4. Salida de la fase

```
{
  "user_profile": {},
  "birth_context": {},
  "preferences_context": {},
  "account_context": {}
}
```

9. FASE 3 — CARGA O CÁLCULO DEL CONTEXTO ASTRAL

9.1. Objetivo

Construir el estado astral necesario para la lectura solicitada.

9.2. Qué debe calcular o recuperar esta fase

Datos natales

- carta natal
- ascendente
- casas
- ángulos
- posiciones planetarias
- aspectos natales
- condición natal resumida

Datos anuales

- edad actual
- profección anual
- casa activada
- Señor del Año
- estado natal del Señor del Año

Datos temporales

- día de la semana
- planeta del día
- Luna actual
- fase lunar
- Sol actual
- rango semanal o mensual si aplica

Datos dinámicos

- tránsitos detectados
- tránsitos relevantes
- tránsitos priorizados
- lunaciones activas
- activaciones de casas y ángulos

9.3. Motores implicados

Esta fase puede alimentarse de:

- `chart_engine.js`

- `ascendant_engine.js`
- `astronomy.js`
- `swisseph`
- `projection_engine.js`
- `transit_engine.js`
- `planetary_engine.js`

9.4. Regla de optimización

a) Reutilizar cálculos cuando sea posible

- carta natal persistida
- estado anual persistente
- caché temporal de Luna o fase lunar si conviene

b) No duplicar cálculos entre vistas

Si ya existe un estado astral del día, la semanal o mensual pueden reutilizar parte de ese contexto.

9.5. Salida de la fase

Un objeto `raw_astral_state` aún no interpretado, pero completo.

10. FASE 4 — NORMALIZACIÓN DE VARIABLES

10.1. Objetivo

Convertir los resultados heterogéneos de la fase astral en un objeto estable, uniforme y compatible con el motor de orquestación.

10.2. Problema que resuelve

Los motores astronómicos pueden devolver datos con formatos distintos. El pipeline necesita una capa de normalización para que el motor interpretativo no dependa del caos técnico.

10.3. Acción

Transformar datos crudos a la estructura definida en el documento 002.

10.4. Estructura esperada

```
{  
  "user_context": {},  
  "natal_context": {},  
  "annual_context": {},  
  "temporal_context": {},  
  "transit_context": {},  
  "output_context": {},  
  "control_context": {}  
}
```

10.5. Ejemplos de normalización

- convertir nombres técnicos a claves internas consistentes
- convertir fechas a formato común
- traducir listas astronómicas a objetos con campos estándar
- convertir cálculos dispersos a variables de sistema

10.6. Salida de la fase

`normalized_kairos_state`

Este objeto debe ser la entrada oficial del motor de orquestación.

11. FASE 5 — VALIDACIÓN DE INTEGRIDAD DEL ESTADO

11.1. Objetivo

Comprobar que el estado normalizado contiene lo necesario para generar la vista solicitada.

11.2. Preguntas que debe responder esta fase

- ¿hay carta natal válida?
- ¿hay contexto anual calculado?
- ¿hay fecha activa válida?
- ¿hay suficientes variables para esta vista?
- ¿falta alguna variable crítica?
- ¿debe activarse fallback?

11.3. Resultado posible

Estado completo

Puede pasar al motor interpretativo normal.

Estado incompleto pero usable

Puede pasar con flags y fallback parcial.

Estado insuficiente

Debe bloquear la lectura completa y entregar salida mínima o mensaje interno de error controlado.

11.4. Salida de la fase

```
{  
  
  "state_valid": true,  
  
  "missing_data_flags": [],  
  
  "fallback_required": false,  
  
  "confidence_precheck": 92  
}
```

12. FASE 6 — DETECCIÓN DE FACTORES ACTIVOS

12.1. Objetivo

Filtrar el estado astral para distinguir qué factores merecen entrar en la interpretación.

12.2. Acción

El sistema debe detectar:

- factores estructurales activos
- activadores fuertes del momento
- moduladores de clima
- factores redundantes
- factores descartables

12.3. Regla central

No todo lo calculado entra en lectura.

12.4. Tipos de salida de esta fase

```
{  
  "active_structural_factors": [],  
  "active_dynamic_factors": [],  
  "active_modulatory_factors": [],  
  "discarded_factors": []  
}
```

12.5. Criterios típicos de descarte

- orbe demasiado amplia
- irrelevancia para la vista
- duplicación temática
- ruido sin valor narrativo
- activación menor sin vínculo estructural

13. FASE 7 — JERARQUIZACIÓN Y RESOLUCIÓN INTERPRETATIVA

13.1. Objetivo

Aplicar el motor 003 para decidir qué pesa más y cómo deben integrarse las tensiones.

13.2. Acción

- scoring de factores
- prioridad estructural
- resolución de conflicto
- condensación interpretativa
- identificación de tema dominante y secundarios

13.3. Regla central

Primero se decide el sentido. Luego se redacta.

13.4. Salida de la fase

```
{  
  "dominant_factor": {},  
  "secondary_factors": [],  
  "resolved_tension_axis": "",  
  "dominant_theme": "",  
  "secondary_themes": [],  
  "intensity_level": "",  
  "guidance_mode": ""  
}
```

13.5. Observación

Esta fase es el corazón funcional del pipeline.
Si aquí falla la jerarquía, todo lo demás sale bonito pero torcido.

14. FASE 8 — SELECCIÓN DE NODOS DE BIBLIOTECA

14.1. Objetivo

Consultar únicamente la parte de la biblioteca estructurada que hace falta para construir la lectura.

14.2. Qué puede consultar

Nodos semánticos

- planeta
- signo
- casa
- fase lunar
- planeta del día
- profección
- Señor del Año

Nodos relacionales

- planeta en signo
- planeta en casa
- Señor del Año por planeta
- casa activada
- Luna en signo
- cruces relevantes

Nodos narrativos

- síntesis breve
- advertencia
- acción
- reflexión
- matrix

- diario

Nodos de reglas

- prioridad
- conflicto
- fallback
- longitud
- coherencia
- personalización

14.3. Regla de consulta

La consulta debe ser:

- mínima
- trazable
- contextual
- determinada por la vista y por los factores priorizados

14.4. Ejemplo

Para una lectura diaria, no tendría sentido cargar toda la biblioteca mensual o anual.

Para una lectura anual, no tendría sentido dar demasiado peso a nodos diarios salvo como apoyo muy secundario.

14.5. Salida de la fase

```
{  
  
  "selected_semantic_nodes": [],  
  
  "selected_relational_nodes": [],  
  
  "selected_rule_nodes": [],  
  
  "selected_output_nodes": []  
}
```

15. FASE 9 — CONSTRUCCIÓN DEL NÚCLEO SEMÁNTICO

15.1. Objetivo

Convertir factores priorizados + nodos de biblioteca en una estructura intermedia de sentido.

15.2. Regla central

Antes de redactar módulos, KAIROS debe saber con claridad:

- de qué va el momento
- dónde está la tensión principal
- qué oportunidad contiene
- qué conviene hacer
- qué conviene observar

15.3. Estructura mínima sugerida

```
{  
  "dominant_theme": "",  
  "secondary_themes": [],  
  "life_area_focus": "",  
  "energy_tone": "",  
  "emotional_climate": "",  
  "main_risk": "",  
  "main_opportunity": "",  
  "recommended_action_axis": "",  
  "reflection_axis": "",  
  "guidance_mode": "",  
  "intensity_level": "",  
  "warning_required": false  
}
```

15.4. Función estratégica

Esta fase desacopla el pensamiento del renderizado.

Eso permite que una misma lectura alimente:

- tarjetas de app
- radar
- matrix
- diario
- notificaciones
- historial

15.5. Salida de la fase

`semantic_core`

16. FASE 10 — ENSAMBLAJE DE MÓDULOS DE SALIDA

16.1. Objetivo

Traducir el núcleo semántico a bloques concretos según la vista solicitada.

16.2. Posibles módulos

- `synthesis`
- `warning`
- `action`
- `reflection`
- `energy_level`
- `matrix`
- `journal_prompt`
- `monthly_focus`
- `weekly_pattern`
- `annual_narrative`
- `notification_text`

16.3. Regla

Los módulos de salida no deben reinterpretar el cielo.
Deben expresar el núcleo ya resuelto por el motor.

16.4. Ejemplos por vista

daily

- síntesis
- advertencia si aplica
- acción
- pregunta

weekly

- síntesis
- foco semanal
- advertencia
- acción
- reflexión

monthly

- proceso central del mes
- áreas activadas
- tensión/oportunidad
- acción orientativa

annual

- narrativa del año
- área estructural
- desafío de maduración
- oportunidad de crecimiento
- orientación estratégica

matrix

- cuerpo
- mente
- espíritu
- nivel de energía

journal

- prompt reflexivo
- eje emocional
- pregunta de profundización

16.5. Salida de la fase

```
{  
  "output_modules": {  
    "synthesis": "",  
    "warning": "",  
    "action": "",  
    "reflection": ""  
  }  
}
```

17. FASE 11 — CONTROL DE TONO, LONGITUD Y LEGIBILIDAD

17.1. Objetivo

Ajustar los módulos a los criterios narrativos del sistema antes de pasar a QA.

17.2. Qué debe controlar esta fase

- tono terapéutico
- claridad del lenguaje
- longitud adecuada a la vista
- ausencia de tecnicismo innecesario
- ausencia de fatalismo
- distinción clara entre módulos
- legibilidad general

17.3. Regla

Aquí se ajusta forma, no sentido.

No debe cambiarse el núcleo interpretativo.

Debe afinarse su expresión.

17.4. Salida de la fase

Módulos narrativamente ajustados y listos para QA.

18. FASE 12 — QA AUTOMÁTICO DE SALIDA

18.1. Objetivo

Verificar que la salida final cumple los criterios mínimos de calidad del sistema.

18.2. Qué debe revisar

- respeto a la jerarquía
- no contradicción
- no fatalismo
- no repetición
- no sobreacción a factores menores
- longitud correcta
- coherencia entre módulos
- coherencia con vistas superiores si aplica

18.3. Resultado posible

passed

La salida puede entregarse.

warning

La salida puede entregarse, pero queda marcada.

failed

La salida debe recomponerse o caer a modo fallback.

18.4. Salida de la fase

```
{  
  "qa_status": "passed",  
  "qa_flags": [],  
  "qa_score": 95  
}
```

19. FASE 13 — PERSISTENCIA E HISTORIAL

19.1. Objetivo

Guardar la ejecución de la lectura para trazabilidad, historial, depuración y coherencia futura.

19.2. Qué conviene persistir

- `user_id`
- `output_type`
- `generated_at`
- `semantic_core`
- módulos de salida
- factores dominantes
- nodos utilizados
- versión de motor
- versión de biblioteca
- QA
- trace id

19.3. Beneficios

- historial de lecturas
- comparación entre días, semanas o meses
- debugging

- auditoría interpretativa
- entrenamiento futuro de mejoras internas

19.4. Regla

Persistir la ejecución, no solo el texto final.

20. FASE 14 — ENTREGA A LA INTERFAZ

20.1. Objetivo

Enviar a la UI una estructura ya lista para renderizar.

20.2. Regla principal

La interfaz no debe interpretar ni recalcular significado.
Solo debe mostrar módulos.

20.3. Formato recomendado de entrega

```
{  
  "meta": {  
    "output_type": "daily",  
    "generated_at": "2026-03-12T10:45:00",  
    "qa_status": "passed"  
  },  
  "summary": {  
    "dominant_theme": "",  
    "intensity_level": ""  
  },  
  "modules": {
```



```
"synthesis": "",  
"warning": "",  
"action": "",  
"reflection": ""  
  
},  
  
"support": {  
  "energy_level": "",  
  "matrix": null  
}  
  
}
```

20.4. Beneficio

Esto mantiene la UI limpia y evita mezclar presentación con lógica.

21. PIPELINE RESUMIDO EN FORMATO SIMPLE

solicitud

↓

carga usuario

↓

cálculo/carga astral

↓

normalización

↓

validación de integridad

↓

detección de factores activos

↓

jerarquización

↓

selección de nodos

↓

núcleo semántico

↓

módulos de salida

↓

control narrativo

↓

QA

↓

persistencia

↓

render en UI

22. RELACIÓN DEL PIPELINE CON FIRESTORE

22.1. Colecciones útiles recomendadas

users

Perfil y configuración del usuario.

natal_charts

Carta natal persistida y validada.

kairos_states

Estados astrales calculados o cacheados.

kairos_library_nodes

Biblioteca estructurada consultable.

kairos_rules

Reglas de prioridad, conflicto, fallback, longitud, etc.

kairos_outputs

Historial de lecturas generadas.

kairos_versions

Versionado de motor, biblioteca y pipeline.

22.2. Regla de arquitectura

Firestore no debe usarse como vertedero desordenado de textos.
Debe servir como soporte estructurado para:

- perfil
- estado
- nodos
- reglas
- historial
- versiones

23. RELACIÓN DEL PIPELINE CON LOS MOTORES JS EXISTENTES

23.1. Motores de entrada/cálculo

- `chart_engine.js`
- `ascendant_engine.js`
- `astronomy.js`
- `projection_engine.js`
- `transit_engine.js`
- `planetary_engine.js`

23.2. Motores de contexto por vista

- `daily_engine.js`
- `weekly_engine.js`
- `monthly_engine.js`
- `radar_engine.js`

23.3. Motores o capas que faltaría ordenar mejor

Sería muy útil centralizar parte del flujo en módulos como:

- `kairos_orchestrator.js`
- `factor_scoring_engine.js`
- `semantic_core_engine.js`
- `qa_engine.js`

No porque suene elegante, sino porque si todo esto vive mezclado en un único archivo grande, luego mantenerlo será una pesadilla con incienso.

24. REGLAS DE DISEÑO DEL PIPELINE

24.1. Una fase no debe invadir la siguiente

Cada fase debe entregar un resultado claro y dejar de hacer cosas que no le pertenecen.

24.2. El pipeline debe ser trazable

Cada respuesta debería poder rastrearse hasta:

- variables usadas
- factores priorizados

- nodos consultados
- versión del sistema

24.3. El pipeline debe ser recuperable

Si una fase falla, el sistema debe poder:

- marcar error
- activar fallback
- evitar colapso general

24.4. El pipeline debe permitir cache y optimización

No todo debe recalcularse cada vez.

24.5. El pipeline debe ser modular

Cada vista puede reutilizar gran parte del mismo flujo sin duplicar lógica.

25. ERRORES QUE ESTE PIPELINE DEBE EVITAR

25.1. Saltarse la normalización

Eso hace que cada módulo lea datos distintos y el sistema se vuelva inconsistente.

25.2. Saltarse el núcleo semántico

Eso produce salidas fragmentadas y repetitivas.

25.3. Dejar que la UI decida

Eso mezcla presentación con pensamiento y destruye orden interno.

25.4. Consultar demasiada biblioteca

Eso ralentiza, dispersa y aumenta contradicción.

25.5. Persistir solo texto final

Eso impide entender después por qué el sistema dijo lo que dijo.

26. CRITERIO DE MANTENIMIENTO

Este documento debe revisarse cuando ocurra cualquiera de estas situaciones:

- cambie el orden de ejecución del sistema
 - se añada una nueva vista
 - cambie la arquitectura de Firestore
 - se incorporen nuevas reglas de QA
 - cambie el modo de selección de nodos
 - se añada una capa LLM de pulido
 - se rehaga la separación entre backend y frontend
-

27. CIERRE DEL DOCUMENTO

El pipeline de generación de respuesta es la columna vertebral operativa de KAIROS.

El 002 define las piezas.

El 003 define cómo piensa el sistema.

Pero el 004 define **cómo todo eso circula y se convierte en una lectura real**.

Sin este documento, KAIROS corre el riesgo de tener:

- cálculo por un lado
- biblioteca por otro
- motores de salida por otro
- y una interfaz bonita que junta cosas sin verdadero orden

Este pipeline existe para impedir eso.

Su función es garantizar que cada respuesta de KAIROS pase por una cadena clara de:

- entrada
- cálculo
- normalización
- selección
- jerarquía
- síntesis
- salida

- validación
- persistencia

En ese orden.

Solo así la app puede ser estable, coherente y escalable.

005_REGLAS_DE_QA_DE_SALIDA

Sistema KAIROS

Criterios de control de calidad para validar la salida interpretativa antes de mostrarla al usuario

1. Propósito del documento

Este documento define las **reglas de QA de salida de KAIROS**, es decir, el conjunto de criterios que deben aplicarse para revisar, validar o bloquear una respuesta antes de que llegue a la interfaz de la app.

Su función es asegurar que toda salida generada por KAIROS cumpla unos estándares mínimos de:

- coherencia
- claridad
- jerarquía interpretativa
- seguridad narrativa
- legibilidad
- consistencia entre vistas
- fidelidad al sistema

KAIROS no debe limitarse a generar textos que “suenen bien”. Debe garantizar que la respuesta:

- esté bien pensada
- esté bien ensamblada
- esté bien expresada
- y no traicione la lógica interna del sistema

Este documento no define qué significa cada factor astrológico ni cómo se construye el núcleo interpretativo. Define **cómo se revisa el resultado final** antes de ser entregado.

2. Función dentro de KAIROS

El QA de salida cumple siete funciones centrales:

1. Detectar fallos de sentido

Identifica contradicciones, exageraciones, desplazamientos indebidos de jerarquía o fragmentación.

2. Detectar fallos narrativos

Evita tecnicismo innecesario, dramatización, vaguedad, repetición o módulos mal diferenciados.

3. Proteger la experiencia del usuario

Asegura que la lectura no sea confusa, alarmista, fría o internamente incoherente.

4. Proteger la identidad del sistema

Garantiza que KAIROS no se degrade hacia un horóscopo superficial o una salida mística vacía.

5. Permitir fallback inteligente

Cuando una salida no alcanza calidad suficiente, el sistema debe poder reducirla, recomponerla o simplificarla.

6. Facilitar trazabilidad

Permite saber qué regla se incumplió, en qué fase y con qué gravedad.

7. Mejorar mantenimiento y depuración

Convierte errores subjetivos en criterios observables y revisables.

3. Alcance

Este documento incluye:

- principios del QA de salida
- categorías de control
- reglas de validación obligatorias
- criterios de aprobación, advertencia o bloqueo
- flags recomendados
- integración con pipeline y persistencia
- criterios de fallback tras fallo de QA

Este documento no sustituye:

- al motor de orquestación
- al pipeline de generación
- a las normas de redacción
- al tono terapéutico
- a las reglas de ensamblaje

Su función es **validar el resultado final del sistema**, no producirlo.

4. Principio general del QA de salida

La regla central del QA en KAIROS es esta:

una salida no es válida solo porque sea gramaticalmente correcta o suene inspiradora; es válida solo si respeta la lógica del sistema, la jerarquía interpretativa y la claridad humana.

Esto implica que una respuesta puede ser rechazada aunque:

- esté bien escrita
- tenga frases bonitas
- parezca profunda
- suene convincente

si:

- contradice la estructura anual
- infla factores menores
- mezcla demasiados temas
- genera confusión
- repite contenido
- introduce fatalismo
- o no guarda coherencia con la vista solicitada

5. Posición del QA dentro del sistema

El QA debe aplicarse **después del ensamblaje de salida y antes de la persistencia final y del render en UI.**

Su posición correcta en el pipeline es:

núcleo semántico

↓

ensamblaje de módulos

↓

ajuste narrativo

↓

QA de salida

↓

persistencia

↓

UI

El QA no sustituye al motor interpretativo.

Actúa como control de calidad posterior, no como generador de significado.

6. Niveles de validación del QA

KAIROS debe trabajar con tres niveles de resultado:

6.1. **passed**

La salida es válida y puede entregarse sin intervención adicional.

6.2. **warning**

La salida es usable, pero presenta alguna debilidad moderada que conviene registrar.

6.3. **failed**

La salida no debe mostrarse tal como está. Debe:

- recomponerse
- simplificarse
- entrar en fallback
- o bloquearse si el problema es crítico

7. Categorías de control de QA

El QA de KAIROS debe revisar al menos estas nueve categorías:

1. coherencia jerárquica
 2. coherencia interna de sentido
 3. coherencia entre vistas
 4. calidad narrativa y tono
 5. claridad y legibilidad
 6. proporcionalidad interpretativa
 7. diferenciación modular
 8. longitud y densidad
 9. trazabilidad y consistencia técnica
-

8. CATEGORÍA 1 — COHERENCIA JERÁRQUICA

8.1. Objetivo

Verificar que la salida respeta la jerarquía interna del sistema y no desplaza factores estructurales por factores menores.

8.2. Regla central

La profección anual, la casa activada y el Señor del Año deben seguir siendo el eje de fondo cuando corresponda. Los factores rápidos solo pueden modular.

8.3. Errores que debe detectar

- la Luna del día define el sentido principal del año
- el planeta del día pesa más que un tránsito mayor
- una fase lunar desplaza un proceso estructural
- un tránsito menor domina la lectura sin justificarlo

8.4. Indicadores de fallo

- el tema dominante proviene de un factor de baja prioridad
- no hay rastro del eje anual en una lectura mensual cuando debería haberlo
- la lectura diaria parece desconectada del ciclo mayor sin razón

8.5. Resultado esperado

- `qa_priority_respected = true`
 - o flag específico si se incumple
-

9. CATEGORÍA 2 — COHERENCIA INTERNA DE SENTIDO

9.1. Objetivo

Verificar que la lectura no se contradiga a sí misma.

9.2. Regla central

Síntesis, advertencia, acción y reflexión deben pertenecer al mismo núcleo interpretativo.

9.3. Errores que debe detectar

- síntesis expansiva y acción restrictiva sin integración
- advertencia alarmista con tono general suave
- reflexión que apunta a otro proceso distinto del descrito
- módulos escritos como si vinieran de lecturas distintas

9.4. Ejemplos de contradicción

Incorrecto

- síntesis: “momento propicio para abrirte y avanzar”
- acción: “conviene retirarte completamente y no moverte”

Correcto

- síntesis: “hay impulso de avance, pero necesita estructura”
- acción: “avanza solo donde ya puedas sostener lo que abres”

9.5. Resultado esperado

- `qa_no_contradiction = true`
-

10. CATEGORÍA 3 — COHERENCIA ENTRE VISTAS

10.1. Objetivo

Asegurar que diaria, semanal, mensual y anual no se contradigan entre sí.

10.2. Regla central

La vista corta puede matizar o focalizar. No debe negar la lógica de la vista más estructural.

10.3. Errores que debe detectar

- la diaria contradice el proceso mensual
- la semanal niega el tono del mes
- la mensual ignora completamente el eje anual
- el diario personal abre un tema que no aparece en ninguna vista principal

10.4. Reglas concretas

Anual

marca la estructura general

Mensual

desarrolla la estructura anual

Semanal

modula el proceso mensual

Diaria

colorea el clima inmediato dentro de ese proceso

10.5. Resultado esperado

- `qa_cross_view_consistency = true`
-

11. CATEGORÍA 4 — CALIDAD NARRATIVA Y TONO

11.1. Objetivo

Verificar que la salida suene como KAIROS y no como un horóscopo, un informe técnico o una pieza de dramatización.

11.2. Regla central

El tono debe ser:

- claro

- humano
- terapéutico
- sobrio
- comprensible
- no fatalista
- no grandilocuente

11.3. Errores que debe detectar

- lenguaje sentencioso
- frases deterministas
- tono amenazante
- exceso de solemnidad
- espiritualidad vacía
- tecnicismo innecesario
- lenguaje frío o mecánico

11.4. Expresiones a evitar

- “vas a sufrir inevitablemente”
- “este tránsito destruirá”
- “todo cambiará para siempre”
- “estás condenado a”
- “cuadratura aplicativa de orbe 1.3” en salida general

11.5. Expresiones preferibles

- “puede sentirse exigente”
- “conviene revisar”
- “es un momento de ajuste”
- “hay presión para ordenar”
- “la intensidad pide consciencia, no dramatización”

11.6. Resultado esperado

- `qa_tone_ok = true`
 - `qa_no_fatalism = true`
 - `qa_no_excess_technicality = true`
-

12. CATEGORÍA 5 — CLARIDAD Y LEGIBILIDAD

12.1. Objetivo

Asegurar que la respuesta sea fácil de comprender sin sacrificar profundidad.

12.2. Regla central

KAIROS debe traducir complejidad, no exhibirla en bruto.

12.3. Errores que debe detectar

- frases demasiado largas
- abstracción excesiva
- exceso de conceptos en una sola frase
- vocabulario técnico innecesario
- falta de eje claro

12.4. Preguntas de control útiles

- ¿se entiende de qué va esta lectura en una primera lectura?
- ¿queda claro cuál es el foco?
- ¿hay una dirección concreta?
- ¿el usuario sabría qué observar o qué hacer?

12.5. Resultado esperado

- `qa_clarity_ok = true`
-

13. CATEGORÍA 6 — PROPORCIONALIDAD INTERPRETATIVA

13.1. Objetivo

Evitar que el sistema exagere, dramatice o infle la importancia de una señal menor.

13.2. Regla central

La intensidad del lenguaje debe corresponderse con la intensidad real del cielo interpretativo.

13.3. Errores que debe detectar

- advertencia fuerte sin señal estructural suficiente
- lectura densa generada por una Luna tensa sin más apoyo
- salida “épica” en contexto de baja señal
- uso de palabras grandes para procesos modestos

13.4. Regla práctica

Señal baja

Salida breve, ligera y útil.

Señal media

Salida enfocada con un matiz claro.

Señal alta

Salida más desarrollada, con advertencia y acción bien integradas.

13.5. Resultado esperado

- `qa_minor_factor_not_overweighted = true`
- `qa_intensity_proportionate = true`

14. CATEGORÍA 7 — DIFERENCIACIÓN MODULAR

14.1. Objetivo

Asegurar que cada bloque de salida cumpla una función distinta.

14.2. Regla central

La síntesis no debe repetir la advertencia.
La advertencia no debe repetir la acción.
La acción no debe duplicar la reflexión.

14.3. Función esperada de cada módulo

synthesis

Resume el sentido principal del momento.

warning

Señala el riesgo o punto de desregulación.

action

Traduce la lectura a orientación práctica.

reflection

Abre una pregunta o eje de consciencia.

matrix

Redistribuye el estado en cuerpo, mente y espíritu.

journal_prompt

Profundiza el eje reflexivo sin inventar otro.

14.4. Errores que debe detectar

- cuatro módulos diciendo lo mismo con distinto maquillaje
- reflexión convertida en consejo
- advertencia convertida en diagnóstico fatalista
- acción formulada como repetición de síntesis

14.5. Resultado esperado

- `qa_module_distinction_ok = true`
-

15. CATEGORÍA 8 — LONGITUD Y DENSIDAD

15.1. Objetivo

Controlar que cada salida tenga el tamaño y densidad adecuados para su vista.

15.2. Regla central

La longitud no debe depender de cuánto material haya disponible, sino de lo que la vista puede sostener sin perder claridad.

15.3. Reglas por vista

notification

Muy breve. Una idea, un tono, una dirección.

daily

Breve. Un eje, un matiz, una acción o reflexión.

weekly

Media. Proceso, advertencia, acción, observación.

monthly

Desarrollada, pero condensada. No ensayo largo.

annual

Más profunda, pero estructurada.

matrix

Breve y claramente segmentada.

journal

Breve, profunda y enfocada.

15.4. Errores que debe detectar

- diaria demasiado larga
- mensual dispersa
- notificación saturada
- annual reducida a algo trivial
- módulos con densidad desigual sin sentido

15.5. Resultado esperado

- `qa_length_ok = true`
 - `qa_density_ok = true`
-

16. CATEGORÍA 9 — TRAZABILIDAD Y CONSISTENCIA TÉCNICA

16.1. Objetivo

Asegurar que la salida puede rastrearse y que no se ha generado desde nodos, reglas o versiones incoherentes.

16.2. Qué debe comprobar

- existe `engine_version`
- existe `library_version`
- existe `generated_at`
- existen `source_nodes_used`
- existe `trace_id`
- la salida pertenece a la vista solicitada
- la respuesta se generó con la versión activa del sistema

16.3. Errores que debe detectar

- nodos ausentes
- mezcla de versiones
- salida sin trazabilidad
- tipo de output incorrecto
- QA imposible de auditar después

16.4. Resultado esperado

- `qa_traceability_ok = true`
-

17. REGLAS OBLIGATORIAS DE APROBACIÓN

Una salida no debe aprobarse como `passed` si falla cualquiera de estas reglas críticas:

1. respeto de jerarquía
2. ausencia de contradicción interna grave
3. ausencia de fatalismo
4. claridad mínima suficiente
5. proporcionalidad interpretativa
6. coherencia con la vista solicitada
7. diferenciación funcional básica de módulos
8. trazabilidad mínima

Si falla una de estas ocho, la salida no debería salir limpia.

18. SISTEMA DE FLAGS RECOMENDADO

KAIROS debería trabajar con flags explícitos de QA. Por ejemplo:

```
{  
  "qa_priority_respected": true,  
  "qa_no_contradiction": true,  
  "qa_cross_view_consistency": true,  
  "qa_tone_ok": true,  
  "qa_no_fatalism": true,  
  "qa_no_excess_technicality": true,  
  "qa_clarity_ok": true,
```

```
"qa_minor_factor_not_overweighted": true,  
"qa_intensity_proportionate": true,  
"qa_module_distinction_ok": true,  
"qa_length_ok": true,  
"qa_density_ok": true,  
"qa_traceability_ok": true  
}
```

Y además:

```
{  
  "qa_status": "passed",  
  "qa_score": 94,  
  "qa_flags": []  
}
```

19. SISTEMA DE PUNTUACIÓN SUGERIDO

Además del aprobado o suspenso, puede ser útil un score interno.

19.1. Escala sugerida

- 90–100 → salida sólida
- 75–89 → salida válida con observaciones menores
- 60–74 → salida usable pero débil; conviene warning o simplificación
- <60 → salida fallida; requiere recomposición o fallback

19.2. Regla

El score no debe sustituir a las reglas críticas.
Una salida no aprueba por tener buena media si incumple una condición grave.

20. RESPUESTA DEL SISTEMA ANTE FALLOS DE QA

20.1. Si el fallo es leve

- mantener salida
- registrar warning
- persistir flags

20.2. Si el fallo es moderado

- simplificar módulos
- reducir longitud
- recomponer advertencia o acción
- reintentar ensamblaje breve

20.3. Si el fallo es grave

- activar fallback
- bloquear salida completa si no hay material fiable
- no mostrar una lectura “bonita pero falsa”

20.4. Regla importante

Ante duda fuerte, KAIROS debe preferir una salida más simple pero honesta antes que una salida inflada y dudosa.

21. QA ESPECÍFICO POR TIPO DE VISTA

21.1. QA de lectura diaria

Debe verificar especialmente:

- brevedad

- foco único
- no sobredramatización
- conexión mínima con estructura anual
- no convertir el clima diario en destino

21.2. QA de lectura semanal

Debe verificar:

- patrón claro
- continuidad con el mes
- acción útil
- no acumulación excesiva de factores

21.3. QA de lectura mensual

Debe verificar:

- presencia del eje anual
- integración de activadores reales del mes
- equilibrio entre profundidad y claridad
- coherencia entre riesgo y oportunidad

21.4. QA de lectura anual

Debe verificar:

- primacía estructural
- narrativa consistente
- no dependencia de moduladores rápidos
- orientación madura y estratégica

21.5. QA de matrix

Debe verificar:

- claridad de segmentación
- correspondencia con núcleo interpretativo
- no contradicción entre cuerpo, mente y espíritu

21.6. QA de journal

Debe verificar:

- profundidad sin dispersión

- pregunta viva y concreta
 - no abrir un tema nuevo desvinculado del núcleo
-

22. RELACIÓN DEL QA CON EL PIPELINE

El QA debe integrarse como fase explícita del pipeline y no como detalle secundario.

Flujo correcto

ensamblaje de módulos

↓

ajuste narrativo

↓

QA

↓

si passed → persistencia + UI

si warning → persistencia + UI + flag

si failed → recomposición o fallback

Regla

La UI nunca debería recibir directamente una salida no validada.

23. RELACIÓN DEL QA CON FIRESTORE Y TRAZABILIDAD

Sería útil guardar en historial al menos:

- qa_status
- qa_score
- qa_flags
- engine_version
- library_version
- trace_id
- source_nodes_used
- fallback_used

Esto os permitirá después:

- auditar errores
 - detectar patrones de fallo
 - refinar reglas
 - comparar versiones del sistema
-

24. ERRORES TÍPICOS QUE ESTE QA DEBE ATRAPAR

24.1. Error de desplazamiento jerárquico

Un factor menor se convierte en eje de lectura.

24.2. Error de inflación narrativa

La salida suena más intensa que el cielo real.

24.3. Error de dispersión

Demasiados temas compiten entre sí.

24.4. Error de duplicación

Los módulos repiten el mismo mensaje.

24.5. Error de contradicción

La acción va en dirección opuesta a la síntesis sin integración.

24.6. Error de tono

El lenguaje se vuelve fatalista, técnico o artificial.

24.7. Error de desconexión entre vistas

La diaria parece escrita por otro sistema.

24.8. Error de opacidad técnica

La salida no puede auditarse después.

25. CRITERIOS DE MANTENIMIENTO

Este documento debe revisarse cuando ocurra cualquiera de estas situaciones:

- aparecen contradicciones repetidas en producción
 - se añaden nuevas vistas de salida
 - se modifica la jerarquía del motor
 - se añaden nuevas familias de nodos narrativos
 - se introduce una capa LLM de pulido
 - cambian los criterios de tono del sistema
 - cambian las reglas de longitud o modularidad
-

26. CIERRE DEL DOCUMENTO

Las reglas de QA de salida son el último filtro de honestidad de KAIROS.

Un sistema puede calcular bien y aun así hablar mal.

Puede sonar profundo y aun así estar diciendo ruido.

Puede tener buena biblioteca y aun así ensamblar una lectura torpe, inflada o contradictoria.

Este documento existe para impedir eso.

Su función es garantizar que cada salida de KAIROS sea:

- fiel a la jerarquía del sistema
- clara para el usuario
- proporcionada a la señal real
- coherente entre módulos

- coherente entre vistas
- narrativamente limpia
- técnicamente rastreable

No es un lujo. Es una defensa contra la deriva.

Porque en una app como KAIROS, el verdadero prestigio no vendrá de sonar intenso, sino de **pensar con orden y hablar con verdad**.

Perfecto. Ahora entramos en **uno de los documentos más importantes de todo el proyecto**, porque es el que convierte **tu biblioteca teórica en algo que la app puede usar realmente**.

Este documento no es de astrología.
Es **arquitectura de datos para KAIROS**.

006_estructura_firestore_para_kairos

Este documento explica:

- cómo se almacenará la biblioteca en Firebase
- cómo la app consultará los documentos
- cómo el motor interpretativo encontrará la información
- cómo dividir los documentos para que sean utilizables.

Piensa esto:

Tus PDFs ahora mismo son **libros humanos**.

Pero la app necesita **piezas pequeñas de información**.

Ejemplo:

Tu documento:

208_significado_senor_del_ano

Firebase lo necesitará así:

collection: kairos_docs

document: 208_significado_senor_del_ano

fields:

titulo

definicion

reglas

dependencias

Este documento define **esa estructura**.

Prompt para el chat paralelo o el agente GPT

(este documento NO necesita Notebook)

Pega este prompt en el **chat paralelo o en tu agente arquitecto**:

Estamos diseñando la arquitectura de datos de una aplicación astrológica llamada KAIROS.

La aplicación utiliza una biblioteca interpretativa organizada en series documentales:

000 serie operativa

100 fundamentos

200 biblioteca base

300 biblioteca relacional

400 salida humana

500 reglas de ensamblaje

600 carta natal

Necesitamos generar un documento técnico que defina cómo esta biblioteca se almacenará dentro de Firebase Firestore para que la aplicación pueda consultarla.

Código del documento:

006_estructura_firestore_para_kairos

Serie:

000 — arquitectura operativa del sistema

Objetivo del documento:

Definir la estructura de base de datos que permitirá almacenar la biblioteca interpretativa de KAIROS dentro de Firebase Firestore.

El documento debe explicar:

- cómo se organizarán las colecciones en Firestore
- cómo se almacenarán los documentos interpretativos
- cómo se dividirán los documentos en campos consultables
- cómo el motor interpretativo consultará la base de datos
- cómo se manejarán dependencias entre documentos

El documento debe incluir las siguientes secciones:

1. Introducción

Explicar por qué la biblioteca interpretativa debe convertirse en estructura de datos.

2. Principios de almacenamiento

Explicar que los documentos de la biblioteca deben almacenarse en formato estructurado y no como PDFs.

3. Estructura general de Firestore

Definir las colecciones principales del sistema, por ejemplo:

kairos_documents

kairos_interpretations

kairos_rules

kairos_user_profiles

4. Estructura de los documentos interpretativos

Cada documento debe contener campos como:

codigo_documento

serie

titulo

objetivo

definiciones_clave

desarrollo

reglas_de_uso

dependencias

version

5. Fragmentación de documentos

Explicar que algunos documentos pueden dividirse en bloques para facilitar su consulta por el motor interpretativo.

6. Consulta de la biblioteca por el motor KAIROS

Explicar cómo el motor interpretativo consulta Firestore para construir interpretaciones.

7. Mantenimiento y versionado

Explicar cómo actualizar documentos sin romper el sistema.

El documento debe ser técnico, claro y estructurado.

No escribir astrología.

El objetivo es definir la arquitectura de datos del sistema KAIROS.

Cómo funcionará realmente la app (te lo explico fácil)

La app hará algo así:

Paso 1

El usuario abre la app.

Paso 2

El sistema calcula:

edad

profección

señor del año

tránsitos actuales

fase lunar

planeta del día

Paso 3

El motor consulta Firestore.

Ejemplo:

leer documento 207

leer documento 208

leer documento 303

leer documento 305

Paso 4

Combina las reglas.

Paso 5

Usa la **serie 400** para generar texto humano.

Visualización simplificada

usuario



motor astronómico



motor interpretativo



consulta firestore



ensamblaje de significado



salida humana

Algo importante que debes entender

Ahora mismo tu proyecto tiene **tres capas**:

1 Biblioteca conceptual

(lo que llevamos semanas creando)

series 100–600

2 Motor interpretativo

reglas

prioridades

combinaciones

3 Infraestructura técnica

firebase

firestore

backend

app

Este documento **006** conecta **la capa 1 con la capa 3**.

Después de esto

El siguiente paso real del sistema será construir:

003_motor_de_orquestacion_interpretativa

Ese documento explicará:

cómo el sistema decide:

qué documentos consultar

en qué orden

cómo combinarlos

Y eso ya es **el cerebro de KAIROS**.

Código del documento

006_estructura_firestore_para_kairos

Título

Estructura Firestore para la biblioteca interpretativa de KAIROS

Objetivo del documento

Definir la arquitectura de datos que permitirá almacenar la biblioteca interpretativa de KAIROS dentro de Firebase Firestore de forma estructurada, consultable y mantenible por el motor interpretativo.

Alcance

Este documento describe:

- la organización general de la biblioteca dentro de Firestore
- la estructura base de las colecciones principales
- el formato de almacenamiento de documentos interpretativos
- la fragmentación de contenido para consulta programática
- el sistema de dependencias y versionado

Este documento no define:

- la lógica interpretativa astrológica
 - las reglas semánticas de ensamblaje
 - la interfaz visual de la app
 - la autenticación de usuarios
-

1. Introducción

La biblioteca interpretativa de KAIROS no puede almacenarse solo como archivos estáticos o documentos largos pensados para lectura humana. Para que una aplicación pueda consultarla, combinarla y reutilizarla en tiempo real, esa biblioteca debe traducirse a una **estructura de datos**.

El objetivo de esta estructura no es solo guardar textos, sino permitir que el sistema:

- encuentre documentos por código, serie o tema
- recupere secciones concretas sin leer el documento completo
- relacione documentos entre sí
- use reglas y bloques interpretativos de forma modular
- mantenga consistencia entre versiones

Firestore es adecuado para este modelo porque permite:

- documentos JSON flexibles
 - lectura rápida por clave
 - subcolecciones
 - indexación
 - escalabilidad por módulos
-

2. Principios de almacenamiento

La biblioteca interpretativa debe almacenarse en formato **estructurado y consultable**, no como PDFs, imágenes o archivos cerrados.

Principios base:

2.1 Formato semántico

Cada documento debe descomponerse en campos con significado funcional, no guardarse como un único bloque de texto plano.

2.2 Lectura por secciones

El motor interpretativo debe poder consultar:

- objetivo
- definiciones
- desarrollo
- reglas
- dependencias

sin necesidad de cargar todo el documento.

2.3 Separación entre contenido y metadatos

El contenido interpretativo y la información técnica del documento deben almacenarse por separado dentro del mismo registro.

2.4 Persistencia estable

Cada documento debe tener un identificador permanente para que otros módulos puedan referenciarlo sin ambigüedad.

2.5 Versionado explícito

Los cambios de contenido no deben sobrescribir sin control versiones anteriores que puedan estar siendo usadas por el sistema.

3. Estructura general de Firestore

Se propone una estructura principal basada en colecciones separadas por función.

3.1 Colecciones principales

kairos_documents

Colección principal de la biblioteca interpretativa.

Contendrá todos los documentos de las series:

- 000
- 100
- 200
- 300
- 400
- 500
- 600

Cada registro representa un documento bibliotecario estructurado.

kairos_document_blocks

Colección para fragmentos o bloques internos de documentos extensos.

Se usará cuando un documento tenga secciones reutilizables o consultables de forma independiente.

Ejemplos:

- fases lunares individuales
 - significados por planeta
 - bloques de salida humana
 - reglas parciales
-

kairos_rules

Colección especializada para reglas operativas y de ensamblaje.

Puede contener una copia indexada de documentos de la serie 500 o reglas extraídas como objetos atómicos para ejecución directa.

kairos_interpretations

Colección para interpretaciones generadas por el motor.

No almacena biblioteca base, sino resultados procesados para usuarios o sesiones.

kairos_user_profiles

Colección para perfiles de usuario, incluyendo:

- datos natales
 - configuración personal
 - preferencias de lenguaje
 - estado del ciclo actual
-

kairos_dependencies

Colección opcional para mapear relaciones entre documentos de forma explícita y consultable.

Útil para validación interna, auditoría y mantenimiento.

3.2 Alternativa mínima

Si se desea una primera versión más simple, puede iniciarse con:

- `kairos_documents`
- `kairos_user_profiles`
- `kairos_interpretations`

y añadir `kairos_document_blocks` y `kairos_rules` en una segunda fase.

4. Estructura de los documentos interpretativos

Cada documento de la biblioteca debe almacenarse como un documento Firestore dentro de `kairos_documents`.

4.1 ID del documento en Firestore

Se recomienda usar como ID:

`codigo_documento`

Ejemplo:

`201_significado_planetas`

Esto evita duplicidad y permite consulta directa.

4.2 Campos base obligatorios

```
{  
  "codigo_documento": "201_significado_planetas",  
  "serie": "200",  
  "serie_nombre": "biblioteca_base",  
  "titulo": "Significado de los planetas",  
  "objetivo": "Definir el significado base de los planetas dentro del sistema KAIROS.",  
  "alcance": "Describe significados fundamentales y no combinaciones.",  
  "definiciones_clave": [  
    {  
      "termino": "planeta",  
      "definicion": "Función psicológica o principio activo dentro del sistema interpretativo."  
    }  
  ],  
  "desarrollo": {  
    "secciones": [  
      {  
        "id_seccion": "1",  
        "titulo": "Definición general",  
        "contenido": "..."  
      },  
      {  
        "id_seccion": "2",  
        "titulo": "Clasificación funcional",  
        "contenido": "..."
```

```

    }
  ]
},
"reglas_de_uso": [
  "Debe utilizarse como documento base.",
  "No sustituye combinaciones relacionales."
],
"dependencias": [
  "202_significado_signos",
  "203_significado_casas"
],
"version": 1,
"estado": "activo",
"idioma": "es",
"fecha_creacion": "2026-03-13T00:00:00Z",
"fecha_actualizacion": "2026-03-13T00:00:00Z",
"autor": "KAIROS",
"tags": [
  "planetas",
  "biblioteca_base",
  "astrologia"
]
}

```

4.3 Campos recomendados

Además de los obligatorios, se recomienda incluir:

- resumen_operativo
- tipo_documento
- nivel_jerarquia
- usa_fragmentacion
- bloques_relacionados
- notas_internas
- fuente_origen
- activo_para_motor

4.4 Estructura sugerida de desarrollo

En vez de guardar un único texto largo, conviene guardar una lista ordenada de secciones.

```
"desarrollo": {
  "secciones": [
    {
      "orden": 1,
      "titulo": "Principio general",
      "contenido": "..."
    },
    {
      "orden": 2,
      "titulo": "Aplicación práctica",
      "contenido": "..."
    }
  ]
}
```

Esto permite:

- renderizado limpio en frontend
- recuperación selectiva
- futura fragmentación automática

5. Fragmentación de documentos

No todos los documentos deben tratarse como una sola unidad. Algunos conviene dividirlos en bloques reutilizables.

5.1 Cuándo fragmentar

Se recomienda fragmentación cuando:

- el documento contiene listas largas de unidades paralelas
- una sección puede consultarse de forma independiente
- el motor necesita acceder a una parte concreta con alta frecuencia
- la salida depende de piezas pequeñas y reutilizables

5.2 Ejemplos de fragmentación útil

Documento 204_significado_fases_lunares

Bloques por fase:

- luna_nueva
- cuarto_creciente
- luna_llena
- cuarto_menguante

Documento 206_significado_planeta_del_dia

Bloques por día:

- domingo_sol
- lunes_luna
- martes_marte

Documento 303_senor_del_ano_por_planeta

Bloques por planeta:

- saturno_como_senor_del_ano
- venus_como_senor_del_ano

5.3 Estructura de bloque

Dentro de `kairos_document_blocks`:

```
{
  "block_id": "206_viernes_venus",
  "documento_padre": "206_significado_planeta_del_dia",
  "serie": "200",
  "tipo_bloque": "unidad_interpretativa",
  "clave": "viernes_venus",
  "titulo": "Viernes — Venus",
  "contenido": {
    "significado_simbolico": "...",
    "estado_psicologico": "...",
    "energia_disponible": "...",
    "acciones_favorables": ["..."],
    "riesgos": ["..."]
  },
  "version": 1,
  "activo": true
}
```

5.4 Regla operativa

El documento completo sigue existiendo en [kairos_documents](#), pero ciertos módulos pueden consultar directamente bloques en [kairos_document_blocks](#).

6. Consulta de la biblioteca por el motor KAIROS

El motor interpretativo no debe leer la biblioteca como si fuera una colección de textos lineales. Debe operar en capas.

6.1 Tipos de consulta

Consulta por código

Para recuperar un documento exacto.

Ejemplo:

- buscar `208_significado_senor_del_ano`

Consulta por serie

Para cargar todos los documentos de una capa concreta.

Ejemplo:

- serie `500` para reglas de ensamblaje

Consulta por tag

Para recuperar documentos relacionados con un tema.

Ejemplo:

- `profecciones`
- `transitos`
- `salida_humana`

Consulta por dependencia

Para reconstruir qué documentos necesita uno específico.

Ejemplo:

- cargar `301_planeta_en_signo` y sus dependencias base

Consulta por bloque

Para recuperar solo la unidad necesaria.

Ejemplo:

- bloque `viernes_venus`

6.2 Flujo de consulta sugerido del motor

Paso 1

Determinar qué capa necesita:

- estructura base
- regla
- bloque de salida humana
- documento relacional
- dato del usuario

Paso 2

Consultar documentos prioritarios por código.

Paso 3

Cargar bloques específicos solo si la lectura lo requiere.

Paso 4

Aplicar reglas de ensamblaje desde la serie 500.

Paso 5

Generar salida humana usando documentos de la serie 400.

6.3 Ejemplo conceptual

Para una lectura diaria:

1. cargar perfil del usuario desde `kairos_user_profiles`
2. identificar casa proyectada y Señor del Año
3. consultar:
 - `304_casa_activada_por_profeccion`
 - `208_significado_senor_del_ano`
 - `305_luna_en_transito_por_signo`
 - `206_significado_planeta_del_dia`
4. consultar reglas:
 - `501_reglas_de_prioridad`
 - `506_reglas_de_consistencia_entre_vistas`
5. cargar bloques de salida:
 - `405_lecturas_diarias`
 - `403_acciones_recomendadas`

7. Mantenimiento y versionado

La biblioteca debe poder evolucionar sin romper el motor interpretativo.

7.1 Versionado obligatorio

Cada documento debe incluir:

- `version`
- `fecha_actualizacion`
- `estado`

Estados sugeridos:

- `draft`
- `activo`
- `archivado`
- `deprecated`

7.2 Regla de actualización

No se debe modificar radicalmente la semántica de un documento activo sin:

- aumentar versión
- registrar cambios
- revisar dependencias

7.3 Compatibilidad interna

Si un documento cambia su estructura, deben revisarse:

- bloques asociados
- reglas que lo consumen
- dependencias declaradas
- prompts o funciones del motor que lo consultan

7.4 Estrategia recomendada

Cambios menores

Actualizar el mismo documento y subir `version`.

Cambios estructurales mayores

Crear nueva versión lógica o nuevo código documental si cambia el propósito.

Ejemplo:

- `301_planeta_en_signo` versión 2 si mejora redacción
- nuevo documento si se divide el alcance

7.5 Auditoría interna

Se recomienda añadir un subregistro o colección auxiliar de cambios:

`kairos_document_changelog`

Ejemplo:

```
{  
  "codigo_documento": "501_reglas_de_prioridad",  
  "version": 2,  
  "fecha": "2026-03-13T00:00:00Z",  
  "cambios": [  
    "Se ajustó el orden entre planeta del día y fase lunar",  
    "Se clarificó prioridad del Señor del Año"  
  ]  
}
```

Recomendación estructural final

Colección principal

`kairos_documents`

Colecciones complementarias

- `kairos_document_blocks`
- `kairos_rules`
- `kairos_interpretations`
- `kairos_user_profiles`
- `kairos_dependencies`
- `kairos_document_changelog`

Principio técnico central

La biblioteca KAIROS debe almacenarse como **base de conocimiento semántica**, no como repositorio de archivos.

Cada documento debe ser:

- identificable
 - consultable por partes
 - versionable
 - relacionable con otros documentos
 - utilizable por el motor interpretativo sin transformación manual
-

Observaciones finales

Esta estructura permite que KAIROS opere con dos niveles simultáneos:

Nivel humano

Los documentos siguen existiendo como piezas editoriales comprensibles.

Nivel algorítmico

Los mismos documentos se convierten en objetos consultables y ensamblables por la aplicación.

La arquitectura Firestore debe sostener ambos niveles sin duplicar lógica ni introducir ambigüedad documental.

Dependencias con otros documentos

Este documento se relaciona especialmente con:

- `005_logica_de_consulta_del_motor_interpretativo`
- serie `500` completa

- toda la serie 200, 300, 400 y 600

Porque define el soporte físico y consultable sobre el que la biblioteca interpretativa podrá funcionar dentro de la aplicación KAIROS.